

NOTROX

Gaming Webshop

FELHASZNÁLÓI DOKUMENTÁCIÓ

Szoftverfejlesztő vizsgaremek / Záródolgozat

Készítette: Péjó Kristóf, Molnár Márk Krisztián, Gróf Richárd László
Intézmény: Budapesti Műszaki SZC Egressy Gábor Két Tanítási Nyelvű Technikum

Weboldal: <https://notrox.hu>

Budapest, 2025–2026

Tartalom

1. Bevezetés.....	3
2. Rendszerkövetelmények és telepítés	5
2.1 Weboldal – Nem szükséges telepítés	5
2.2 WPF asztali alkalmazás – Telepítés	5
2.2.1 Szükséges NuGet csomagok (fejlesztői célra)	5
2.3 Avalonia mobilalkalmazás – Telepítés.....	6
2.3.1 Android.....	6
3. A weboldal használata – Vásárlói útmutató	6
3.1 Regisztráció	6
3.1.1 Regisztráció lépései.....	6
3.1.2 Regisztrációs hibaüzenetek	7
3.2 Bejelentkezés.....	7
3.2.1 Bejelentkezési folyamat	7
3.3 Profiladatok kezelése.....	8
3.3.1 Adatok módosítása	8
3.3.2 Jelszó megváltoztatása	8
3.3.3 Profilkép feltöltése	9
3.4 Szállítási és számlázási cím kezelése	9
4. Termékek böngészése és vásárlás	11
4.1 A termékek oldal	11
4.2 Termékek kosárba helyezése.....	11
4.3 A kosár kezelése.....	12
4.4 Rendelés leadása	13
4.4.1 Szállítási adatok ellenőrzése.....	13
4.4.2 Bankkártya adatok megadása	13
4.5 Sikeres rendelés visszaigazoló oldal	14
5. Korábbi rendelések és számla	15
5.1 Rendeléstörténet megtekintése	15
5.2 Rendelési állapotok	15
6. A weboldal többi oldala	16
6.1 Főoldal (/).....	16
6.2 Rólunk oldal (/aboutus)	17
6.3 Kapcsolat oldal (/contact).....	17
6.4 Adatvédelmi tájékoztató (/adatvedelmi)	18
6.5 ÁSZF (/aszf)	18
6.6 404-es oldal (/404)	19

7. WPF asztali alkalmazás – Adminisztrátori útmutató	20
7.1 Az alkalmazás megnyitása	20
7.2 Felhasználók kezelése	20
7.3 Rendelések kezelése	21
7.4 Termékek kezelése	22
7.5 Keresési funkció	22
7.6 CSV exportálás.....	23
7.7 Termék hozzáadása	23
8. Avalonia mobilalkalmazás – Adminisztrátori útmutató.....	24
8.1 Bejelentkezés.....	24
8.2 Navigáció az Avalonia alkalmazásban.....	25
8.3 Termékek kezelése és módosítása.....	26
8.4 Rendelések menedzselése	27
8.5 Új termék rögzítése a rendszerben	28
8.6 Mobilspecifikus viselkedés	29
9. Webes Adminisztrációs felület bemutatása.....	29
9.1 Adminisztrátori bejelentkezés	29
9.2 Felhasználók kezelése	30
9.3 Terméklista és termék módosítás	31
9.4 Rendelések felügyelete.....	32
9.5 Rendelések felügyelete.....	33
9.6 Kijelentkezés (logout)	33
10. Hibaelhárítás és GYIK	34
10.1 Weboldal – Gyakori problémák	34
A rendelés gomb inaktív marad	34
Nem látok termékeket a termékek oldalon.....	34
A profilkép nem frissül feltöltés után.....	34
A bejelentkezés nem működik a WPF alkalmazásban.....	34
10.2 Kapcsolat és ügyfélszolgálat	35
11. Összefoglalás.....	35
Felhasznált irodalom és forrásmegjelölés	36

1. Bevezetés

Üdvözljük a NOTROX gaming platform felhasználói dokumentációjában! Ez a kézikönyv részletesen bemutatja a platform három kliensalkalmazásának telepítési és használati útmutatóját: a webalapú webáruházat (notrox.hu), a Windows rendszerre szánt WPF asztali alkalmazást, valamint az Android eszközökre elérhető Avalonia mobilalkalmazást.

A dokumentáció két fő felhasználói csoportnak szól: vásárlóknak, akik a weboldalon keresztül böngésznek és rendelnek, és adminisztrátoroknak, akik a WPF vagy Avalonia alkalmazáson keresztül kezelik a rendszer tartalmát.

A NOTROX webáruház gaming perifériákat forgalmaz – egereket, billentyűzeteket, fejhallgatókat, egérpadokat, mikrofonokat és gamer székeket. A weboldal URL-je: <https://notrox.hu>.

2. Rendszerkövetelmények és telepítés

2.1 Weboldal – Nem szükséges telepítés

A NOTROX weboldalhoz semmilyen telepítés nem szükséges. Az oldal böngészőből érhető el a <https://notrox.hu> URL-en. Az alábbiakban az ajánlott rendszerkövetelmények találhatók:

Komponens	Minimális	Ajánlott
Böngésző	Chrome 90+, Firefox 88+, Edge 90+, Safari 14+	Legújabb stabil verzió
Képernyőfelbontás	1024×768 px	1920×1080 px
Internetkapcsolat	5 Mbps	20 Mbps+
JavaScript	Engedélyezve	Engedélyezve (kötelező)

2.2 WPF asztali alkalmazás – Telepítés

A WPF alkalmazás Windows 10 vagy Windows 11 rendszert igényel. A futtatáshoz szükséges a .NET 8 (vagy újabb) futtatókörnyezet megléte. Az alábbi lépések alapján telepíthető az alkalmazás:

1. Töltse le a legújabb kiadást a projekt GitHub repository-jából (Releases szekció).
2. Csomagolja ki az archívumot egy tetszőleges mappába.
3. Ellenőrizze, hogy telepítve van-e a .NET 8 Desktop Runtime:
<https://dotnet.microsoft.com/en-us/download/dotnet/8.0>
4. Futtassa a Notrox.exe fájlt.
5. Az első indításkor szükség lehet a szervercím konfigurálására (alapértelmezett: <https://notrox.hu>).

2.2.1 Szükséges NuGet csomagok (fejlesztői célra)

Csomag	Verzió	Felhasználás
CommunityToolkit.Mvvm	8.x	MVVM parancsok és property értesítés
Microsoft.Extensions.DependencyInjection	8.x	Függőség befecskendezés
System.Text.Json	Beépített .NET 8	JSON szerIALIZÁCIÓ

2.3 Avalonia mobilalkalmazás – Telepítés

2.3.1 Android

6. Engedélyezze az ismeretlen forrásból való telepítést az Android beállításában.
7. Töltse le az .apk fájlt a GitHub Releases oldaláról.
8. Nyissa meg a letöltött .apk fájlt és kövesse a telepítési varázsló utasításait.
9. Engedje meg az alkalmazás számára az internethasználati engedélyt.

3. A weboldal használata – Vásárlói útmutató

3.1 Regisztráció

A webáruház teljes funkcionalitásához regisztrált fiókra van szükség. A regisztráció ingyenes és néhány percet vesz igénybe. A regisztrációs oldal a jobb felső sarokban található 'Regisztráció' gombra kattintva, vagy a /register URL-en érhető el.

NOTROX

Termékek Rólunk Kapcsolat

Login Register

REGISZTRÁCIÓ

FELHASZNÁLÓNÉV

Válassz egy nevet

EMAIL CÍM

pelda@email.com

JELSZÓ

Legalább 8 karakter

JELSZÓ MEGERŐSÍTÉSE

Jelszó megerősítése

☐ Elfogadom a Felhasználási Feltételeket és az ÁSZF-et

FIÓK LÉTREHOZÁSA

Már tag vagy? [Jelentkezz be itt!](#)

3.1.1 Regisztráció lépései

10. Nyissa meg a <https://notrox.hu/register> oldalt.
11. Adja meg a felhasználónevet (Username mező – kötelező).
12. Adja meg az e-mail címét (Email mező – kötelező, @ jelet kell tartalmaznia).
13. Adja meg a jelszavát (Password mező – legalább 8 karakter).
14. Pipálja be az ÁSZF elfogadása jelölőnégyzetet (kötelező a regisztrációhoz).

15. Kattintson a 'Regisztráció' gombra.

16. Sikeres regisztráció esetén a rendszer a bejelentkezési oldalra irányítja.

3.1.2 Regisztrációs hibaiüzenetek

Szituáció	Megjelenített üzenet	Megoldás
Hiányzó @ jel az e-mail mezőben	Kérjük adjon meg érvényes e-mail címet (alert)	Helyes e-mail formátum: felhasznalo@domain.hu
ÁSZF nincs bepipálva	El kell fogadnia az ÁSZF-et a regisztrációhoz (alert)	Pipálja be az ÁSZF jelölőnégyzetet
Már regisztrált e-mail	Van már ilyen felhasználó (szerver üzenet)	Próbáljon más e-mail címmel, vagy lépjen be
Jelszó túl rövid	Legalább 8 karakteres jelszó szükséges	Adjon meg legalább 8 karakteres jelszót
Üres mezők	A gomb inaktív marad (disabled állapot)	Töltse ki az összes kötelező mezőt

3.2 Bejelentkezés

Ha már rendelkezik felhasználói fiókkal, a bejelentkezési oldal a /login URL-en érhető el. A navigációs sávban a 'Bejelentkezés' gombra kattintva is odanavigálhat.

NOTROX

Termékek Rólunk Kapcsolat

Login Register

BEJELENTKEZÉS

FELHASZNÁLÓNÉV

Írd be a felhasználóneved

JELSZÓ

Elfelejtetted a jelszót?

BEJELENTKEZÉS

Nincs még fiókod? [Regisztrálj most!](#)

TERMÉKEK: Egér

ADATVÉDELEM: ADATVÉDELMI TÁJÉKOZTATÓ

TELEFONSZÁM: +36201111111

Megnyitás a Térképben

3.2.1 Bejelentkezési folyamat

17. Nyissa meg a <https://notrox.hu/login> oldalt.

18. Adja meg regisztrált felhasználónevét a 'Felhasználónév' mezőbe.

19. Adja meg jelszavát a 'Jelszó' mezőbe.

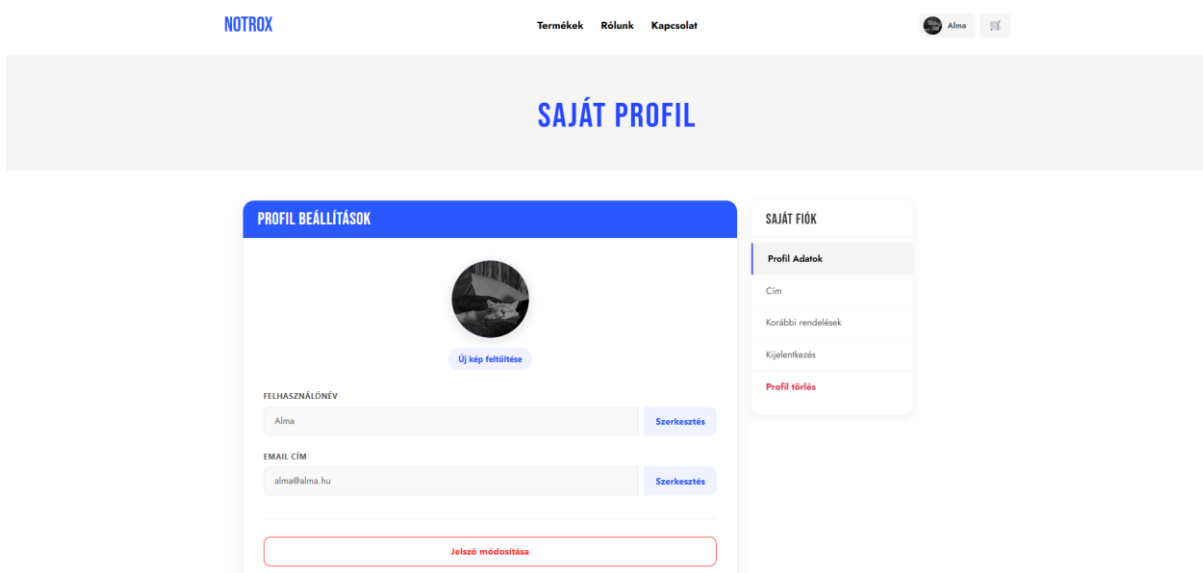
20. Kattintson a 'Bejelentkezés' gombra.

21. Sikeres bejelentkezés után a rendszer a profiloldalra irányítja, és a navigációban megjelennek a bejelentkezett felhasználónak szóló menüpontok.

Szituáció	Megjelenített üzenet	Megoldás
Helytelen jelszó	Hibás felhasználónév vagy jelszó (alert)	Ellenőrizze a jelszót, vagy állítsa vissza
Nem létező felhasználónév	A felhasználó nem található (alert)	Ellenőrizze a felhasználónevet vagy regisztráljon
Bejelentkezett felh. megnyitja a login oldalt	Automatikus átirányítás a profiloldalra	–

3.3 Profiladatok kezelése

A profiloldal a /profil URL-en érhető el. Ide csak bejelentkezett felhasználók juthatnak be; bejelentkezés nélküli hozzáférési kísérlet esetén a rendszer a 404-es oldalra irányít.



3.3.1 Adatok módosítása

22. Navigáljon a profiloldalra (/profil).
23. Kattintson a módosítani kívánt mező melletti 'Szerkesztés' gombra.
24. Módosítsa az adatot (felhasználónév vagy e-mail cím).
25. Kattintson a pipa ikonra a mentéshez.

3.3.2 Jelszó megváltoztatása

Jelszó módosítása

Jelenlegi jelszó

Új jelszó

Új jelszó megerősítése

Változtatások mentése

- 26.
27. Görgessen le a 'Jelszó módosítása' szekcióhoz.
28. Kattintson a 'Jelszó módosítása' piros gombra – megjelennek a jelszómezők.
29. Adja meg jelenlegi jelszavát.
30. Adja meg az új jelszót, majd ismétlje meg.
31. Kattintson a Mentés gombra.

3.3.3 Profilkép feltöltése

32. A profilolalon kattintson a profilkép alatti 'Kép módosítása' gombra.
33. Válasszon JPG, JPEG vagy PNG formátumú képfájlt.
34. A rendszer automatikusan feltölti és megjeleníti az új profilképet.

3.4 Szállítási és számlázási cím kezelése

CÍMEK KEZELÉSE

SZÁLLÍTÁSI CÍM

VÁROS
Alma

IRÁNYÍTÓSZÁM
1167

CÍM 1
Thomas Shelby utca 07.

SZÁMLÁZÁSI CÍM

VÁROS (SZÁMLÁZÁSI)
Budapest

IRÁNYÍTÓSZÁM (SZÁMLÁZÁSI)
1167

CÍM 1 (SZÁMLÁZÁSI)
Thomas Shelby utca 07.

Címek mentése

SAJÁT FIÓK

Profil Adatok

Cím

Korábbi rendelések

Kijelentkezés

Profil törlés

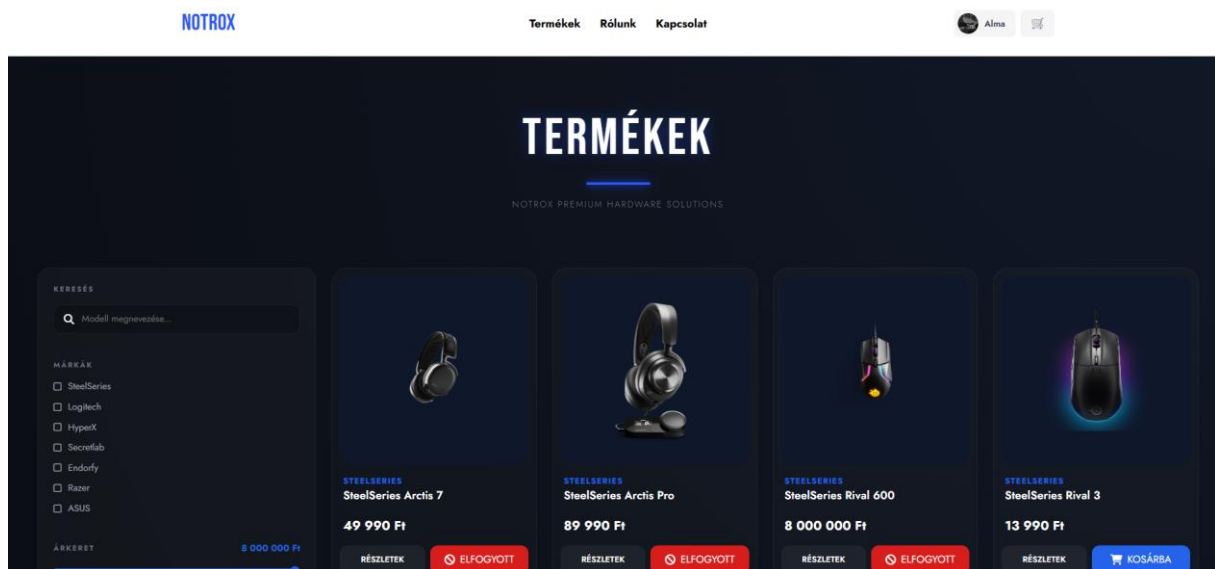
A szállítási cím megadása kötelező a rendelés leadásához. A cím a /address oldalon adható meg.

35. Navigáljon a profilmenüből a 'Szállítási adatok' menüpontra (/address).
36. Töltse ki a Város, Irányítószám és Utca/házzám mezőket.
37. Kattintson a 'Mentés' gombra.
38. Ha a számlázási cím eltér a szállítási címtől, töltse ki a számlázási cím mezőket is.

4. Termékek böngészése és vásárlás

4.1 A termékek oldal

A termékek a /products URL-en böngészhetők. Az oldal az adatbázisból dinamikusan tölti be az összes elérhető terméket. Minden termék egy kártyán jelenik meg, amelyen látható a kép, a megnevezés, a gyártó és az ár.



4.2 Termékek kosárba helyezése

39. Böngésszen a /products oldalon.
40. A kívánt terméknel kattintson a 'Kosárba' gombra.
41. Megjelenik egy visszajelzés (toast vagy alert), hogy a termék a kosárba került.
42. A kosár ikonon a navigációban látható a kosárban lévő elemek száma.

Fontos: A kosár adatai a böngésző localStorage-ában felhasználó-specifikusan (cart_{userId}) tárolódnak. Ez azt jelenti, hogy a kosár megmarad az oldal bezárása után is, de eszközönként és böngészőnként eltér.

4.3 A kosár kezelése

KOSÁR



A kosarad meg üres.

[Irány a termékhez](#)

RENDELÉSI ADATOK

SZÁLLÍTÁSI ADATOK

Alma

1167 Thomas Shelby utca 67.

☒ Ingyenesen átvesszük a szállítást.

FIZETÉSI ADATOK

Kártya / Bankkártya / Utalás

☐ Készlet ellenőrzés

1000 Ft / 2000 Ft / 3000 Ft

Nettó végösszeg: 0 Ft
ÁFA (27%): 0 Ft
Bruttó összesen: 0 Ft

[MEGRENDELÉS →](#)

KOSÁR



STEELSERIES
SteelSeries QcK Mousepad
6990 Ft / db

– 1 +

6990 Ft ×



LOGITECH
Logitech G903
49 990 Ft / db

– 3 +

149 970 Ft ×



HYPERX
HyperX Cloud II
34 990 Ft / db

– 1 +

34 990 Ft ×

RENDELÉSI ADATOK

SZÁLLÍTÁSI ADATOK

Alma

1167 Thomas Shelby utca 67.

☒ Ingyenesen átvesszük a szállítást.

FIZETÉSI ADATOK

Kártya / Bankkártya / Utalás

☐ Készlet ellenőrzés

1000 Ft / 2000 Ft / 3000 Ft

Nettó végösszeg: 151 142 Ft
ÁFA (27%): 40 808 Ft
Bruttó összesen: 191 950 Ft

[MEGRENDELÉS →](#)

Funkció	Leírás	Hogyan?
Mennyiség növelése	Adott termékből több darabot rendelni	+ gomb a terméknel
Mennyiség csökkentése	Kevesebb darabot rendelni	– gomb (0 esetén automatikusan törlődik)
Termék törlése	Termék eltávolítása a kosárból	× gomb a terméknel
Összesítő	Bruttó ár, nettó ár (27% ÁFA), ÁFA összege	Automatikusan frissül
Maximális készletellenőrzés	Ha a készlet számon felül próbál mennyiséget növelni	Alert: 'Ebből a termékből nincs több raktáron'

4.4 Rendelés leadása

A rendelés leadásához be kell jelentkezni, és szállítási cím szükséges a profil beállításokban. A kosár oldalon (/cart) a következő lépések szerint tud rendelést feladni:

4.4.1 Szállítási adatok ellenőrzése

A kosár oldalon a jobb oldalon megjelenik a korábban megadott szállítási cím. Ha még nem adott meg szállítási címet, egy figyelmeztető üzenet jelenik meg, és a Fizetés gomb inaktív marad.

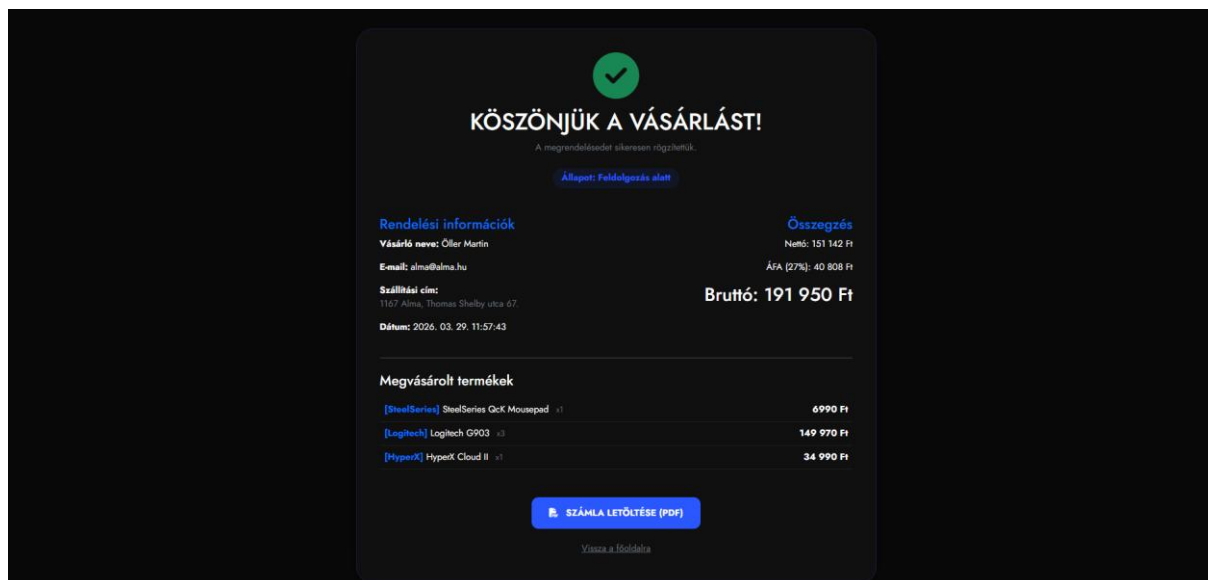
4.4.2 Bankkártya adatok megadása

43. Töltse ki a 'Kártyabirtokos neve' mezőt.
44. Adja meg a 16 számjegyű kártyaszámot (automatikusan formázza 4-es csoportokba).
45. Adja meg a lejárat dátumot MM/ÉÉ formátumban (automatikus validáció: hónap 01–12).
46. Adja meg a CVV/CVC kódot (3 számjegy).
47. Kattintson a 'Fizetés' gombra.

Szituáció	Üzenet / Kimenet
Sikeres rendelés	Alert: 'Sikeres vásárlás!' + átirányítás /success oldalra
Üres kosár	Alert: 'Üres a kosarad!'
Nincs szállítási cím	Alert: 'Rendelés előtt kérjük rögzítse szállítási címét'
Hiányos bankkártya adatok	Alert: 'Kérlek add meg a bankkártya adatokat'
Hiányos szállítási adatok	Alert: 'Kérlek töltsd ki a szállítási adatokat'

4.5 Sikeres rendelés visszaigazoló oldal

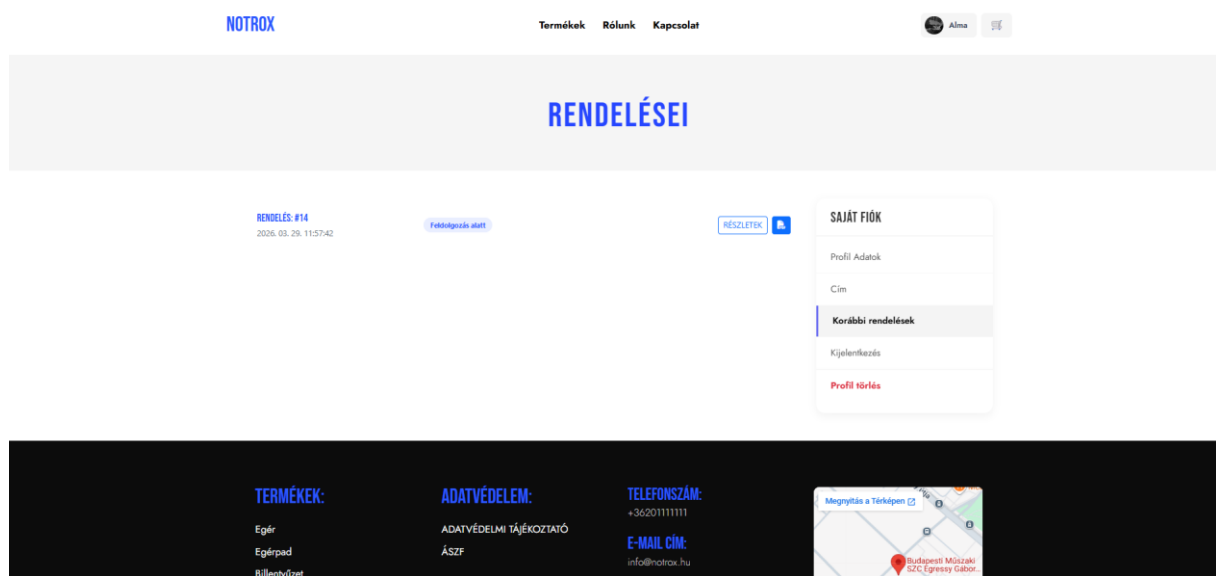
A rendelés feladása után a /success oldal töltődik be, amely tartalmazza a rendelés összes részletét: a rendelés azonosítóját, a megrendelt termékek listáját, a szállítási adatokat, a végösszeget és a rendelés dátumát.



5. Korábbi rendelések és számla

5.1 Rendeléstörténet megtekintése

A korábbi rendelések a /recentorders oldalon tekinthetők meg. Ide csak bejelentkezett felhasználók juthatnak be. Az oldal listázza az összes leadott rendelést, a rendelés azonosítójával, dátumával, állapotával és végösszegével.



48. Navigáljon a profilmenüből a 'Korábbi rendelések' menüpontra.
49. Az oldal betöltésekor automatikusan megjelennek az összes korábbi rendelés.
50. Kattintson egy rendelésnél a 'Részletek' gombra a részletes nézet megtekintéséhez.
51. A PDF ikon gombra kattintva letöltheti a rendelés számláját PDF formátumban.

5.2 Rendelési állapotok

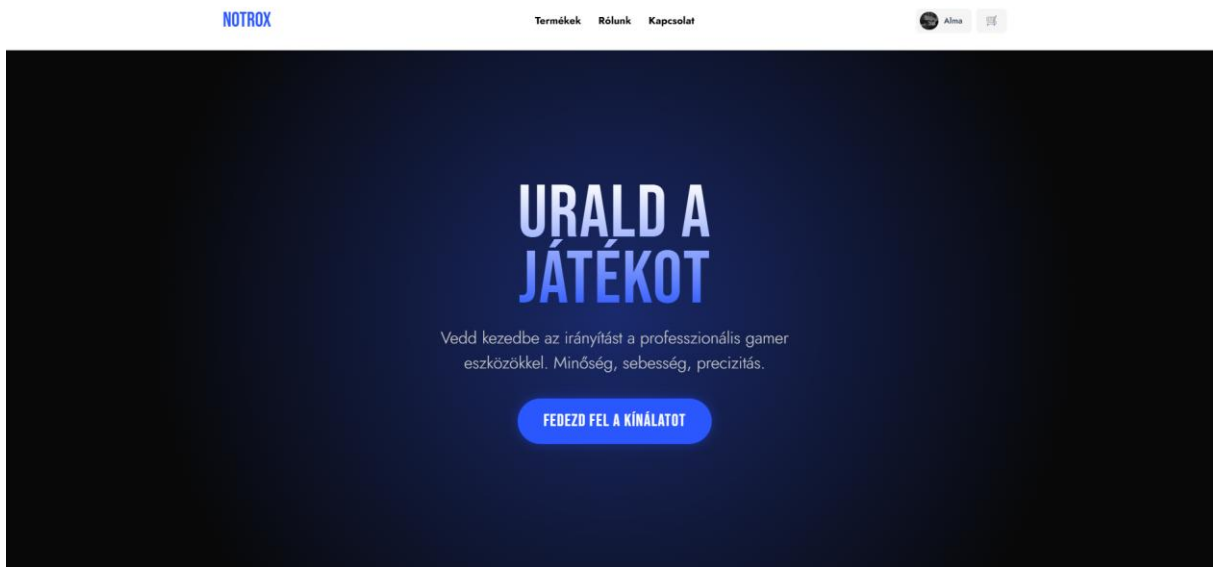
Állapot	Leírás
Feldolgozás alatt	A rendelést felvettük, feldolgozás folyamatban
Összekészítés alatt	A rendelést elkezdjük összekészíteni
Csomagolás kész	A csomag becsomagolásra került
Átadva futárnak	A csomag átadva a futárnak
Kiszállítás alatt	A csomag kiszállítás alatt van
Kiszállítva	A csomag sikeresen kiszállításra került

Állapot	Leírás
Törölve	A rendelés törlésre került

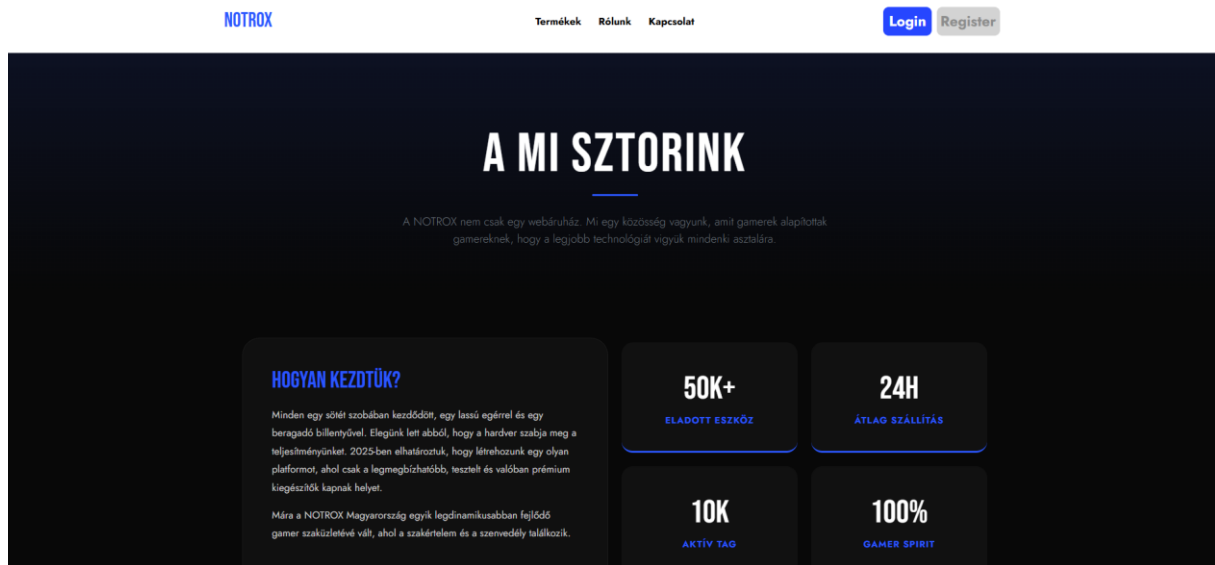
6. A weboldal többi oldala

6.1 Főoldal (/)

A főoldal a NOTROX webáruház landing page-je. Bemutatja a termékkategóriákat (egerek, billentyűzetek, fejhallgatók), az értékJánlatokat (gyors szállítás, garancia, gamer support), és egy CTA (call-to-action) gombot a termékekhez navigáláshoz.

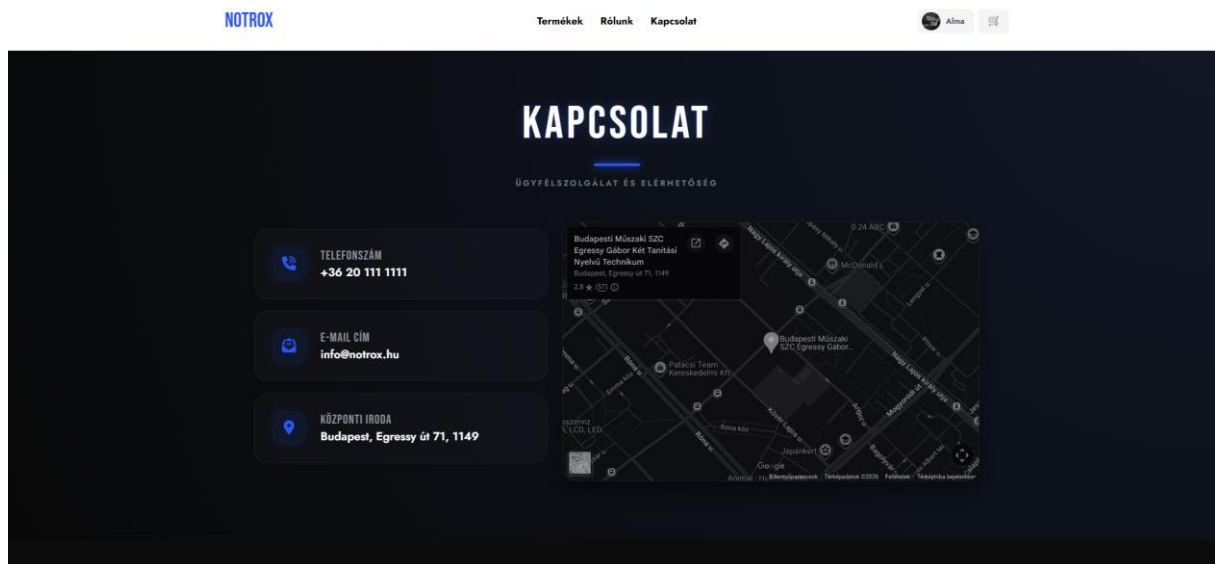


6.2 Rólunk oldal (/aboutus)

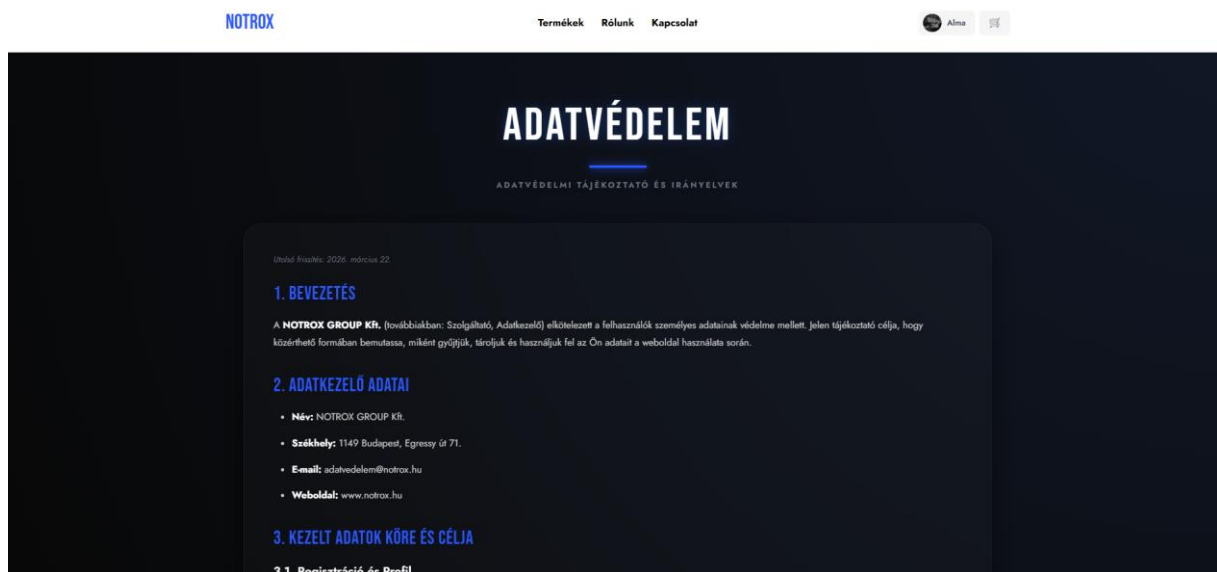


6.3 Kapcsolat oldal (/contact)

A kapcsolat oldal tartalmazza a cég elérhetőségeit (telefonszám, e-mail, cím) és egy beágyazott Google Maps térképet, amely megmutatja az iroda helyzetét: Budapest, Egressy út 71, 1149.

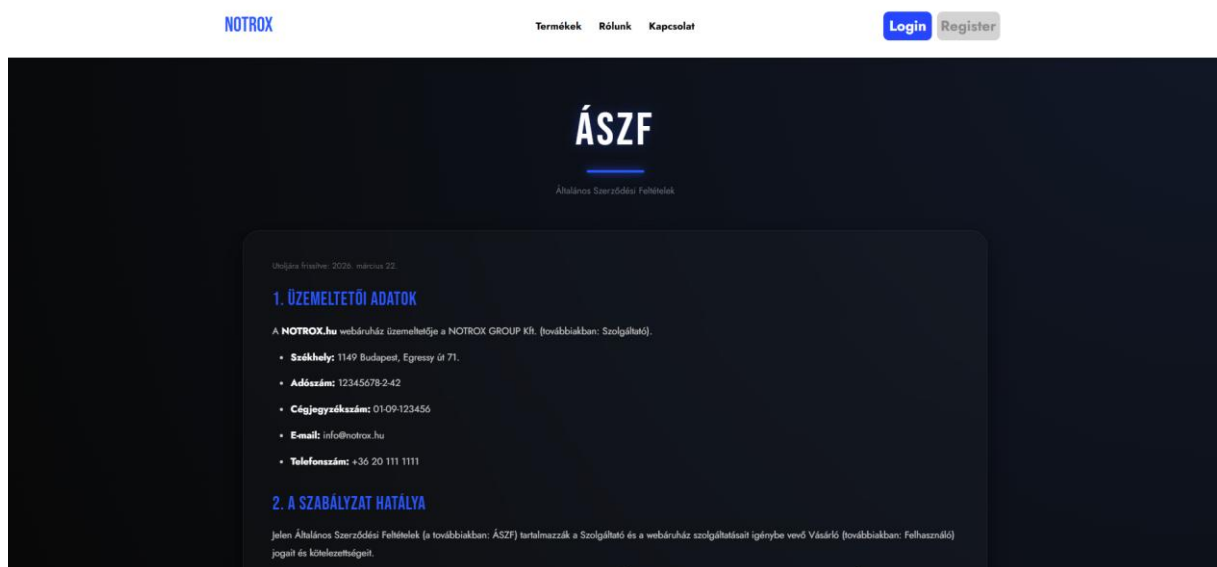


6.4 Adatvédelmi tájékoztató (/adatvedelmi)

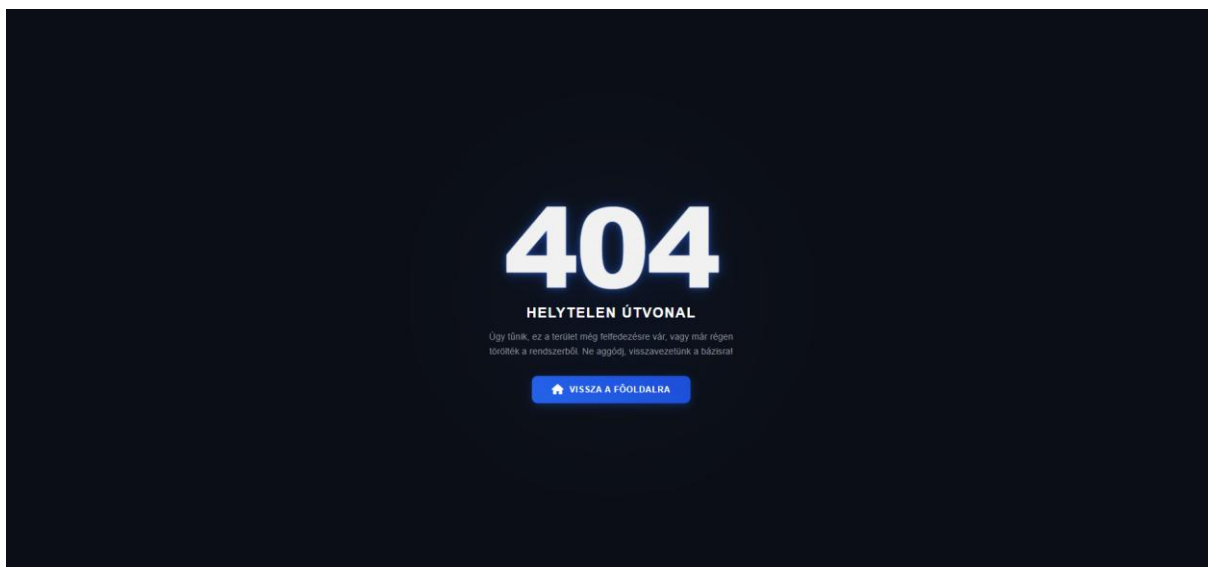


Az adatvédelmi tájékoztató leírja, hogyan kezeli a NOTROX platform a felhasználók személyes adatait, milyen célokból gyűjti azokat és milyen jogaik vannak az érintetteknek az adataikkal kapcsolatban.

6.5 ÁSZF (/aszf)



6.6 404-es oldal (/404)

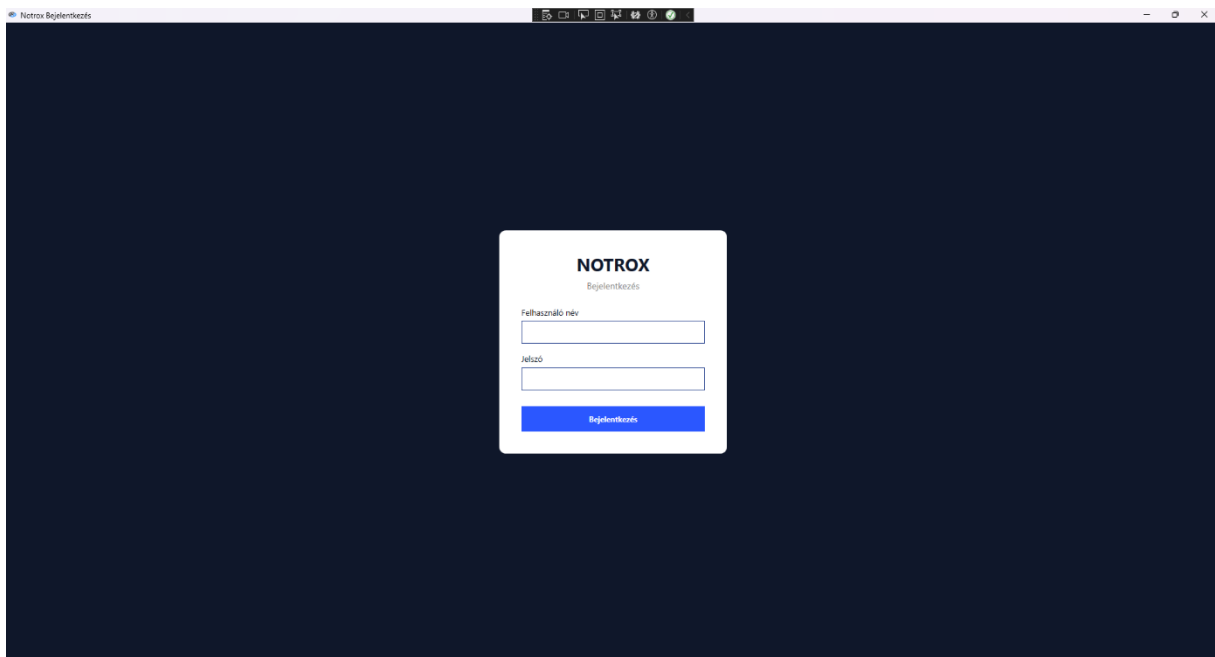


Ha egy felhasználó olyan oldalra próbál navigálni, amelyhez nincs jogosultsága (pl. bejelentkezés nélkül a profiloldalra), vagy amely nem létezik, a rendszer automatikusan a 404-es hibaoldalra irányítja.

7. WPF asztali alkalmazás – Adminisztrátori útmutató

7.1 Az alkalmazás megnyitása

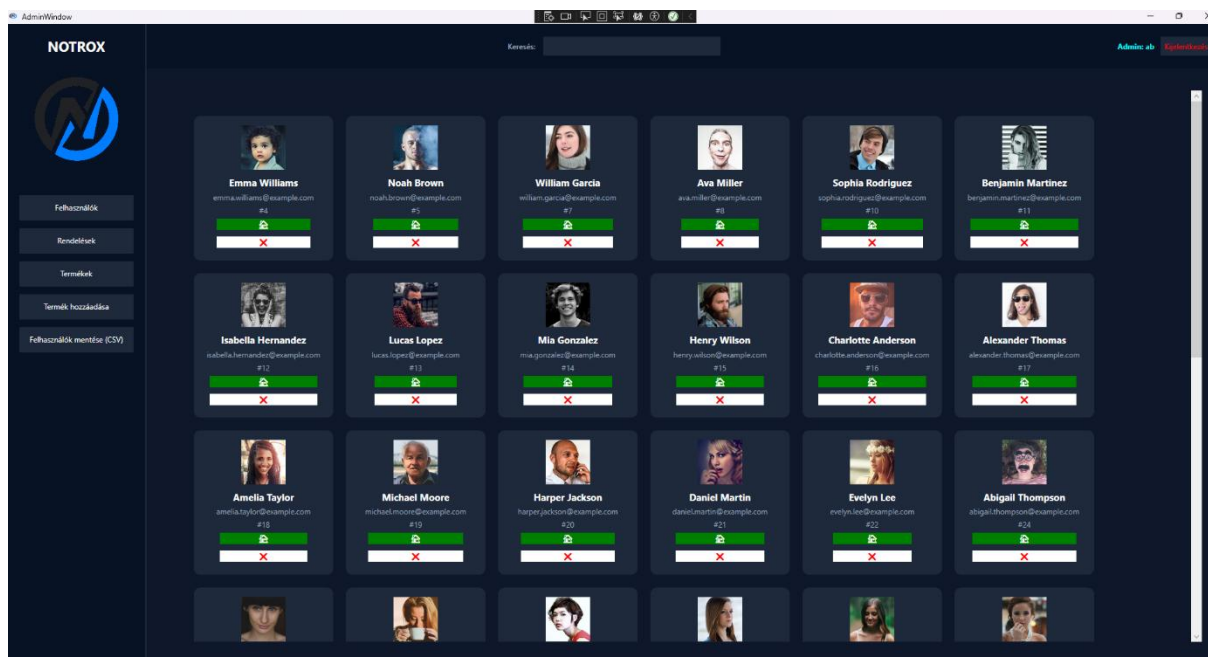
52. Futtassa a Notrox.exe fájlt.
53. A megnyíló ablakban adja meg admin felhasználónevét és jelszavát.
54. Kattintson a 'Bejelentkezés' gombra.
55. Sikeres bejelentkezés után megnyílik az admin főablak.



7.2 Felhasználók kezelése

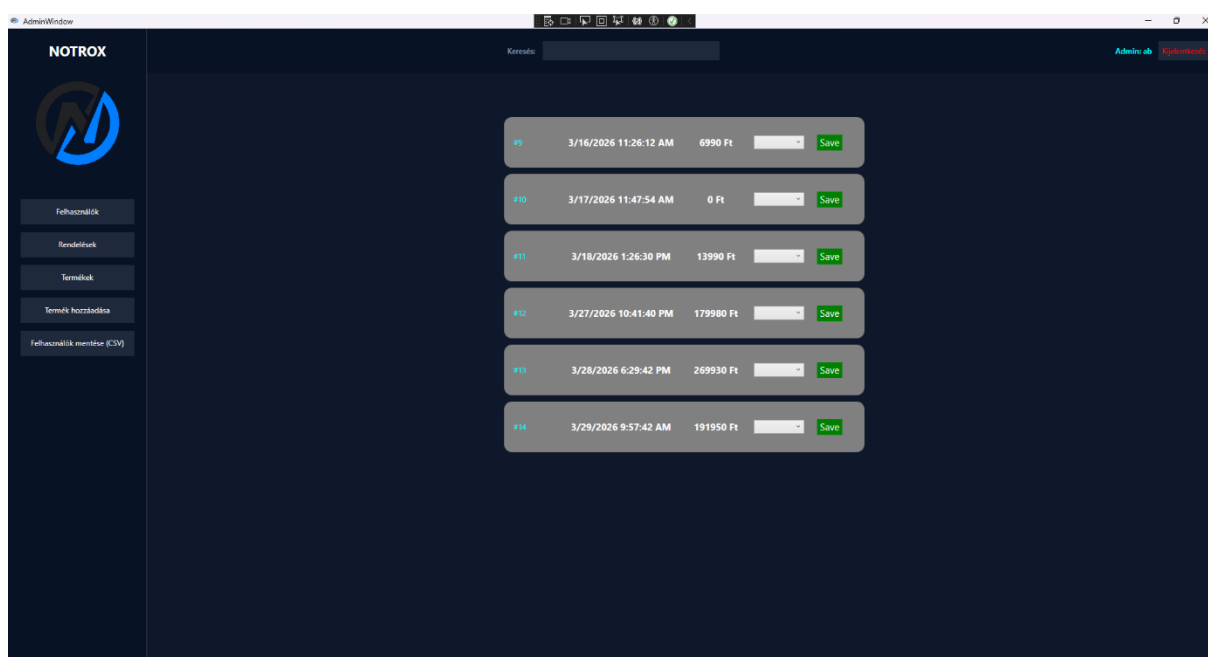
A bal oldali navigációs sávban a 'Felhasználók' gombra kattintva megjelenik az összes regisztrált felhasználó listája. Az egyes felhasználóknál megjelenik a neve, e-mail címe és admin státusza.

Funkció	Leírás	Hogyan?
Felhasználók listázása	Az összes regisztrált felhasználó megjelenítése	Automatikusan betöltődik
Felhasználó törlése	Felhasználói fiók és adatai törlése	Törlés gomb az adott sornál
Keresés	Felhasználók szűrése névsoron alapján	A fejlécben lévő keresőmezőbe írva



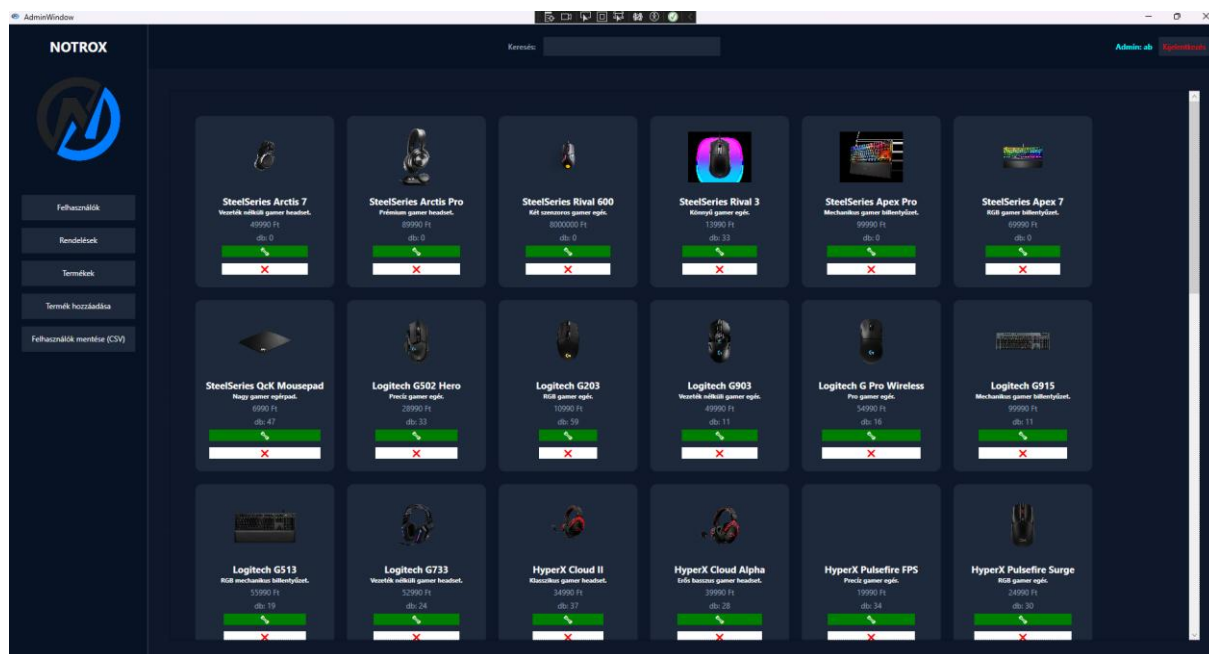
7.3 Rendelések kezelése

A 'Rendelések' nézet listázza az összes rendelést, a felhasználó adataival, a rendelés dátumával és állapotával. Az admin módosíthatja a rendelési fázist (pl. 'Feldolgozás alatt' → 'Teljesítve').



7.4 Termékek kezelése

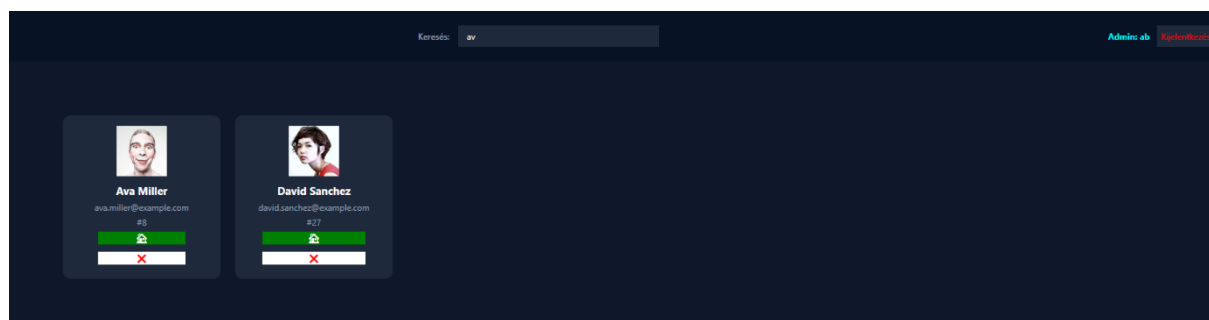
A 'Termékek' gombra kattintva megjelenik az összes termék. Innen lehetőség van szerkeszteni és törölni a meglévő termékeket. Új termék hozzáadásához az 'Új termék' gomb megnyomásával nyílik meg az AddProductWindow dialógusablak.



Funkció	Leírás
Termék szerkesztése	Megnyílik az EditProductsWindow, ahol módosítható a név, leírás, ár, készlet, kép URL
Termék törlése	Megerősítő dialog után törlődik az adatbázisból
Új termék	Megnyílik az AddProductWindow, ahol kitölthető az összes adat

7.5 Keresési funkció

Az admin ablak fejlécében egy globális keresőmező található. A keresési szöveg valós időben szűri az aktuálisan aktív nézet tartalmát (felhasználók, termékek vagy rendelések). A szűrés az ISearchable interfész megvalósításán keresztül működik minden nézetben.

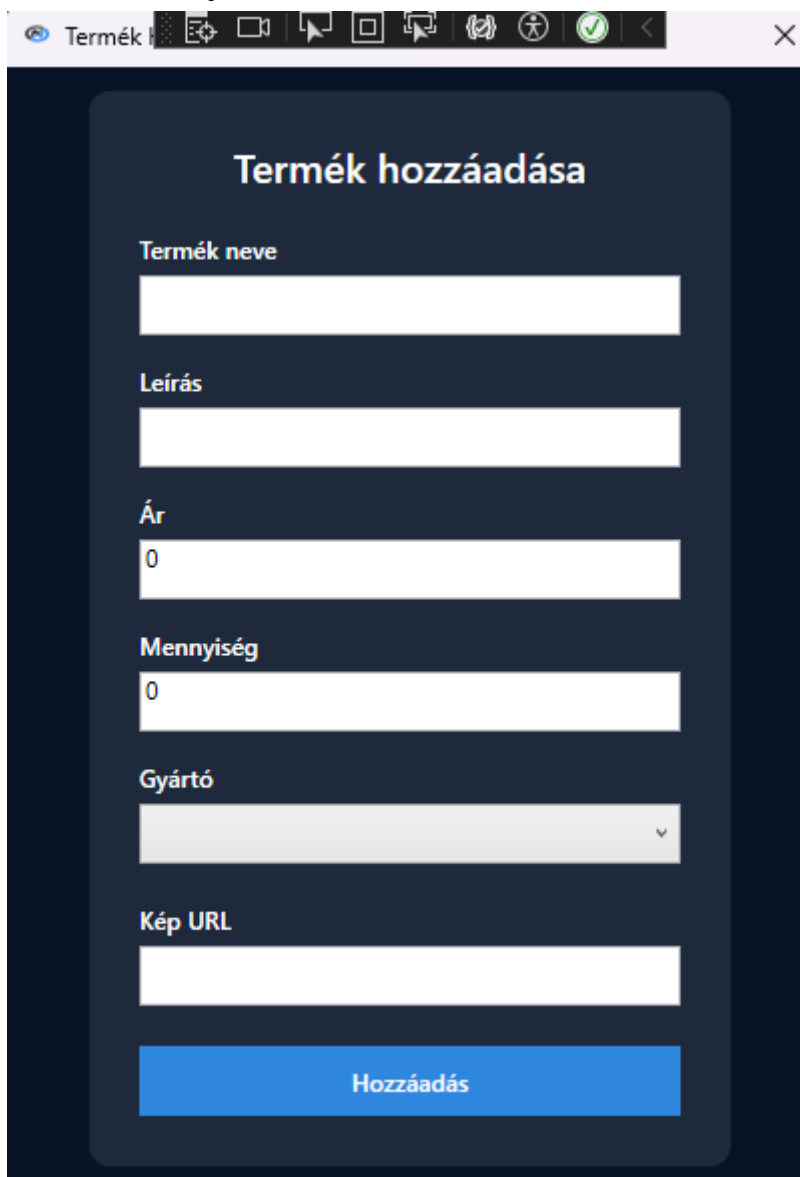


7.6 CSV exportálás

A 'CSV exportálás' gombra kattintva a jelenlegi nézet adatai CSV formátumban kerülnek exportálásra. Megjelenik egy fájlmentés dialógusablak, ahol kiválasztható a célmappa és a fájlnev.

7.7 Termék hozzáadása

A 'Termék hozzáadása' gombra kattintva előjön egy ablak melyben az adatok megadása után létre tud hozni új termékeket.



Termék hozzáadása

Termék neve

Leírás

Ár

0

Mennyiség

0

Gyártó

Kép URL

Hozzáadás

8. Avalonia mobilalkalmazás – Adminisztrátori útmutató

Az Avalonia mobilalkalmazás funkcionálisan megegyezik a WPF asztali alkalmazással, azonban platformfüggetlen megvalósítással. Az alkalmazás elérhető Android, iOS és Desktop platformokon.

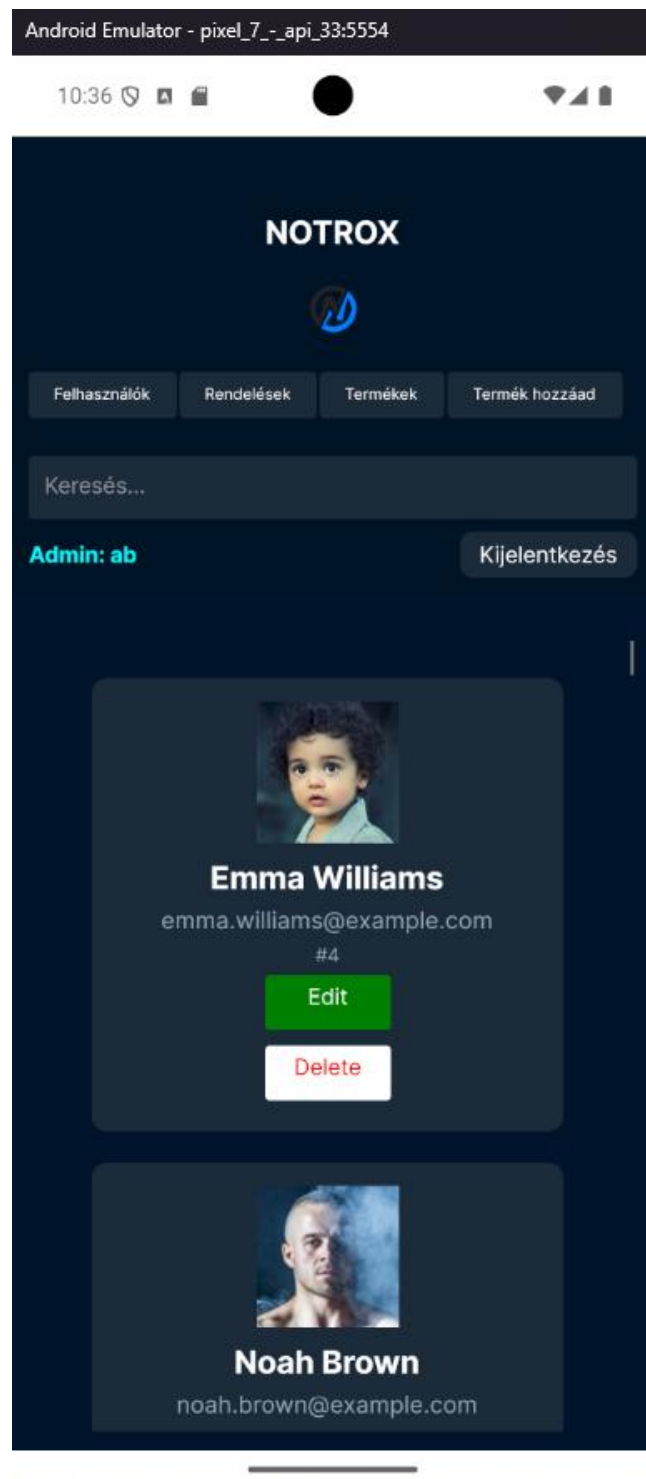
8.1 Bejelentkezés

- 56. Nyissa meg az alkalmazást.
- 57. A StartUpView megnyitásakor adja meg admin felhasználónevét és jelszavát.
- 58. Kattintson a 'Bejelentkezés' gombra.
- 59. Sikeres bejelentkezés után az AdminView jelenik meg.



8.2 Navigáció az Avalonia alkalmazásban

Az Avalonia alkalmazás navigációja a MainViewModel CurrentView property cseréjén alapul. A navigációs gombok az AdminView oldalán helyezkednek el: Felhasználók, Termékek, Rendelések, Cím szerkesztés, Termék szerkesztés, Termék hozzáadás.

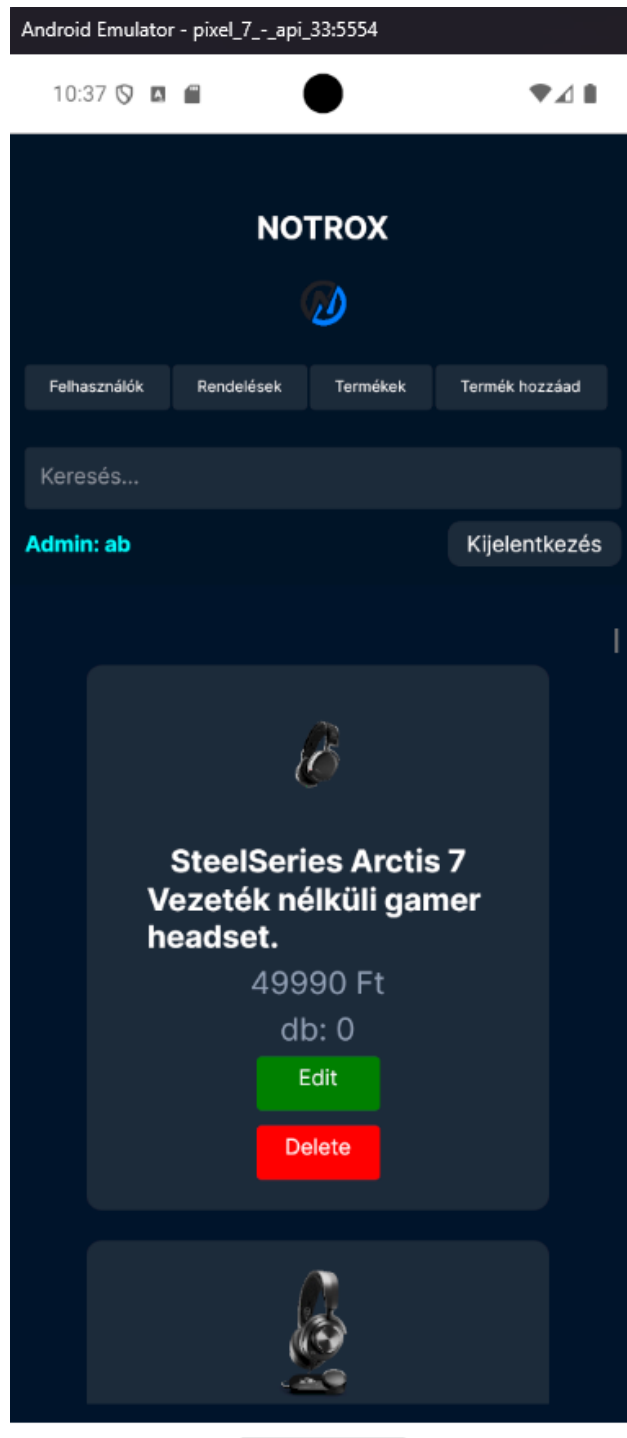


8.3 Termékek kezelése és módosítása

A **Terméklista** oldalon az adminisztrátor valós időben tekintheti át a teljes árukészletet.

Minden termék kártyáján két interaktív művelet érhető el:

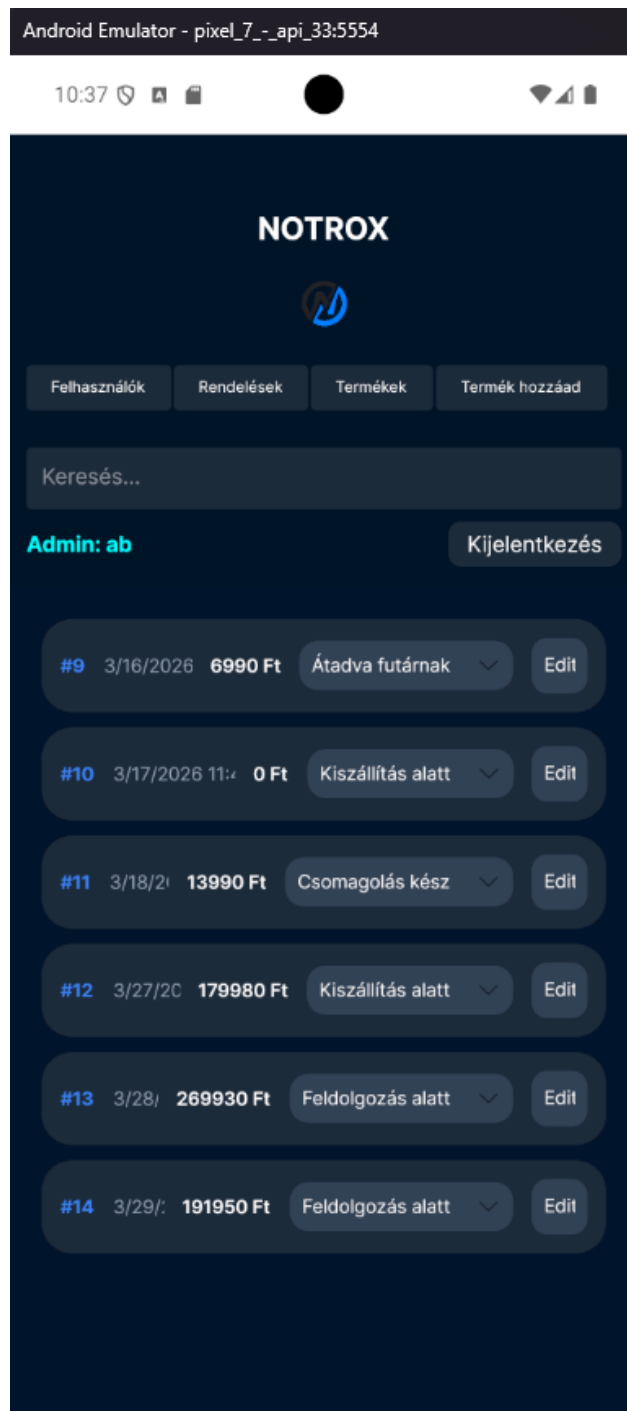
- **Szerkesztés:** A zöld színű **(Edit) gombra** kattintva a tétel adatai (név, márka, leírás, ár, készlet és kép URL) szerkeszthetővé válnak.
- **Törlés:** A piros **(Delete) gomb** aktiválásával kitörli az adott terméket az oldalról és az adatbázisból is.



8.4 Rendelések menedzselése

A **Rendeléskezelő** felületen az összes beérkezett vásárlás listázásra kerül, növekvő sorrendben az azonosító alapján. Az adminisztrátornak lehetősége van a rendelések állapotának nyomon követésére és frissítésére:

- A sor végi **(Edit)** gomb **rakattintva** a rendelés aktuális fázisa (**Phase**) módosíthatóvá válik.
- Egy legördülő menü segítségével az adminisztrátor átállíthatja a csomag állapotát (például: *Csomagolás kész*, *Átadva futárnak* vagy *Kiszállítva*), így a felhasználó a saját profilján mindig a legfrissebb információkat látja a rendeléséről.



8.5 Új termék rögzítése a rendszerben

A **Termék hozzáadása** menüpont alatt egy strukturált űrlap segítségével bővíthető a webshop kínálata. A sikeres létrehozáshoz az alábbi adatok megadása kötelező:

- A termék pontos megnevezése és rövid szakmai leírása.
 - A fogyasztói ár (Ft) és a kezdeti raktárkészlet mennyisége.
 - A gyártó (Company) kiválasztása egy dinamikus legördülő listából.
 - Egy érvényes kép URL megadása a vizuális megjelenítéshez.
- A rendszer validálja az űrlapot, és csak minden mező kitöltése után engedélyezi a rögzítést az adatbázisba.

The screenshot shows the NOTROX mobile application interface. At the top, the status bar indicates the time is 10:38. The app header displays the NOTROX logo and a navigation bar with buttons for 'Felhasználók', 'Rendelések', 'Termékek', and 'Termék hozzáad'. Below the navigation bar is a search bar labeled 'Keresés...'. The user is logged in as 'Admin: ab', and there is a 'Kijelentkezés' (Logout) button. The main content area is titled 'Termék hozzáadása' and contains a form with the following fields: 'Termék neve' (Product name), 'Leírás' (Description), 'Ár' (Price) with a value of 0, 'Mennyiség' (Quantity) with a value of 0, 'Gyártó' (Manufacturer) with a dropdown menu, and 'Kép URL' (Image URL). A blue button at the bottom of the form is labeled 'Termék létrehozása' (Create Product).

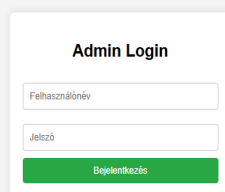
8.6 Mobilspecifikus viselkedés

Jellemző	Android	iOS
Navigáció	Vissza gomb kezelt	Swipe gesture
Klaviatúra	Automatikusan felcsúszik	Automatikusan felcsúszik
Képernyőméret	Skálázható layout	Skálázható layout
Hálózat	Engedély szükséges	NSAppTransportSecurity konfiguráció

9. Webes Adminisztrációs felület bemutatása

9.1 Adminisztrátori bejelentkezés

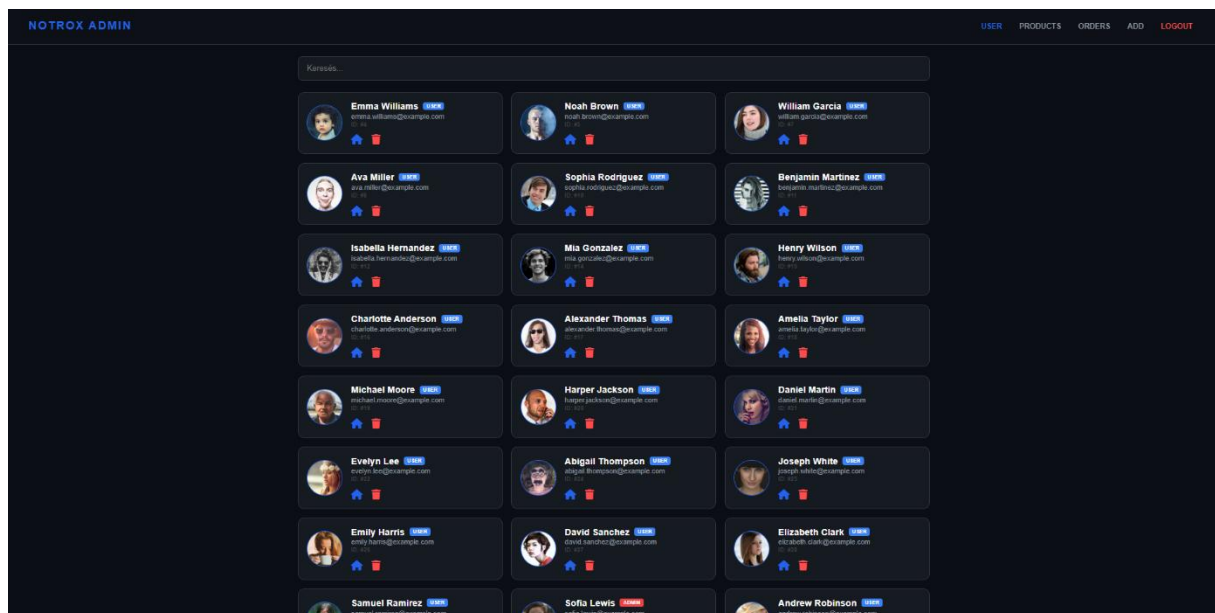
Az adminisztrátori felület elérése csak és kizárólag a notrox.hu/admin útvonalon keresztül történik. A rendszer kiemelt biztonsági ellenőrzést alkalmaz: a bejelentkezés csak akkor sikeres, ha a megadott hitelesítési adatok helyesek, és a felhasználó az adatbázisban is `isAdmin = true` jogosultsággal rendelkezik. A sikeres bejelentkezés után a rendszer generál egy JWT tokenet, amelyet a `sessionStorage` tárol a munkamenet végéig.

A screenshot of the Admin Login form. It is a white box with a light gray border. At the top, it says "Admin Login". Below that, there are two input fields: "Felhasználónév" (Username) and "Jelszó" (Password). At the bottom, there is a green button with the text "Bejelentkezés" (Login).

9.2 Felhasználók kezelése

Az adminisztrátor az elsődleges panelen láthatja az összes regisztrált felhasználót. Az átláthatóságot egy valós idejű (real-time) kereső segíti, amely név vagy e-mail cím alapján szűri a listát.

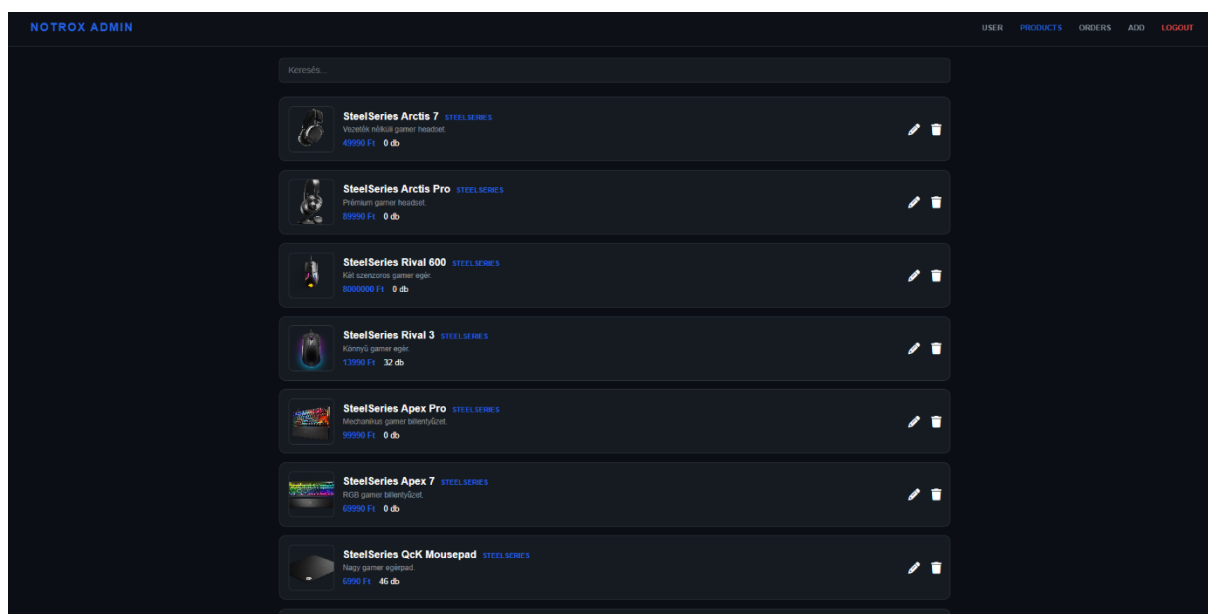
- Címadatok megtekintése: A kék házikonra kattintva egy modern, Glassmorphism stílusú ablakban jelennek meg a felhasználó mentett szállítási és számlázási címei.
- Felhasználó törlése: A piros kuka ikonra kattintva a rendszer egy megerősítést kér. A véletlen törlés elkerülése érdekében a „DELETE” szó begépelése kötelező a művelet véglegesítéséhez.



9.3 Terméklista és termék módosítás

A /product-list oldalon a webshop teljes kínálata kezelhető egy elegáns listában. Minden sor tartalmazza a termék képét, nevét, márkáját, aktuális árát és a raktárkészletet.

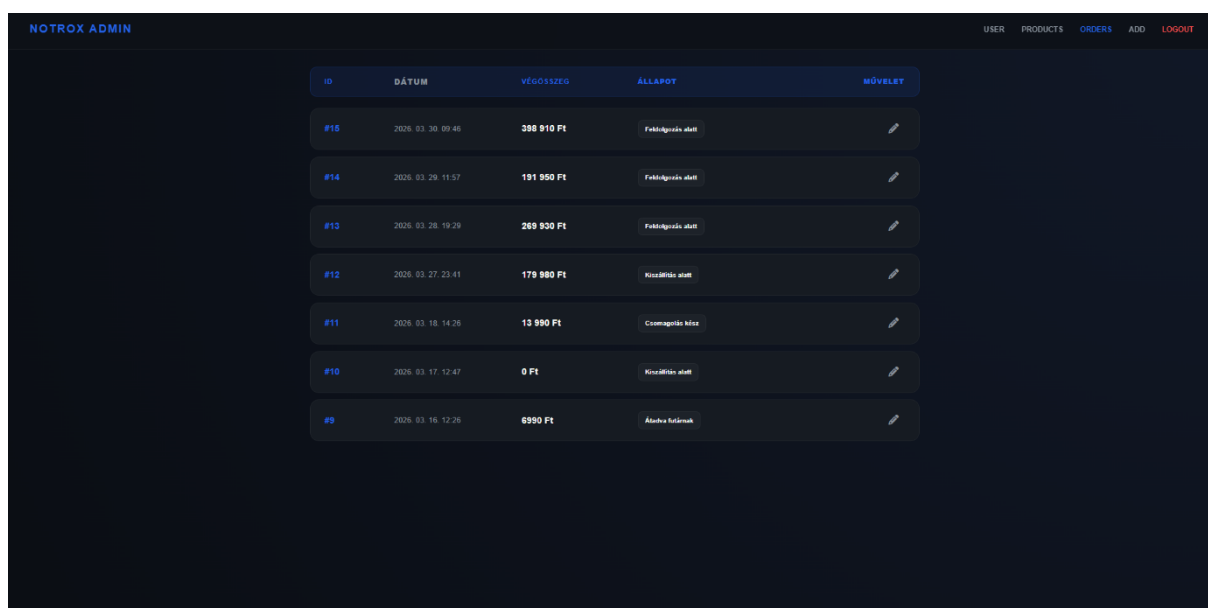
- **Inline szerkesztés:** A ceruza ikonra kattintva az adatok beviteli mezőkké alakulnak. Ekkor a ceruza helyett egy zöld pipa jelenik meg; erre kattintva a rendszer aszinkron (PUT) kéréssel frissíti az adatbázist és a felületet is.
- **Termék törlése:** Hasonlóan a felhasználókezeléshez, a termékek is törölhetők a kuka ikon segítségével, egy megerősítő modallal kiegészítve.



9.4 Rendelések felügyelete

Az adminisztrátor a /order-list oldalon látja az összes leadott megrendelést, alapértelmezés szerint időrendben csökkenő sorrendben (a legfrissebb felül). A felület mobilon is reszponzív, kártya alapú nézetre vált.

- Státusz frissítése: A rendelés „Phase” (fázis) értéke a ceruza ikonnal módosítható. Egy legördülő menüből választható ki a csomag aktuális állapota (pl. Csomagolás kész, Kiszállítva), ami azonnali visszajelzést ad a vásárlónak is.



ID	DÁTUM	VÉGÖSSZEGET	ÁLLAPOT	MŰVELET
#15	2026. 03. 30. 09:46	398 910 Ft	Futóábránd alatt	
#14	2026. 03. 29. 11:57	191 950 Ft	Futóábránd alatt	
#13	2026. 03. 28. 19:29	269 930 Ft	Futóábránd alatt	
#12	2026. 03. 27. 23:41	179 980 Ft	Kiszállítás alatt	
#11	2026. 03. 18. 14:36	13 990 Ft	Csomagolás kész	
#10	2026. 03. 17. 12:47	0 Ft	Kiszállítás alatt	
#9	2026. 03. 16. 12:26	6990 Ft	Állapot frissítve	

9.5 Rendelések felügyelete

A kínálat bővítése a /product-add oldalon lehetséges. Az űrlap dinamikusan kéri le az adatbázisból a létező gyártókat (Company), így az adminisztrátor egy legördülő menüből választhatja ki a márkát. A rendszer csak akkor engedélyezi a „Mentés” gombot, ha minden kötelező mező (név, leírás, ár, készlet, kép URL) helyesen ki van töltve.

9.6 Kijelentkezés (logout)

A biztonságos munkavégzés érdekében az adminisztrátori felület minden oldalán, a fejléc jobb szélén helyet kapott egy feltűnő, piros színnel megjelölt LOGOUT gomb. A termék pontos megnevezése és rövid szakmai leírása.

- A folyamat leírása: A gombra kattintva a rendszer azonnal végrehajtja a kijelentkezési protokollt. A háttérben futó JavaScript függvény törli a böngésző tárolóiból (sessionStorage és localStorage) az érvényes JWT token-t és a felhasználó egyedi azonosítóját (uid).
- Biztonsági hatás: Ezzel a lépéssel a munkamenet érvénytelenné válik, így az auth-check.js védelmi mechanizmusa a továbbiakban nem engedélyezi a hozzáférést a védett oldalakhoz.
- Átírányítás: A sikeres kijelentkezést követően a szoftver automatikusan visszairányítja a felhasználót az adminisztrátori bejelentkezési oldalra (/admin), megakadályozva ezzel, hogy illetéktelenek a böngésző "Vissza" gombjával hozzáférjenek az érzékeny adatokhoz.

10. Hibaelhárítás és GYIK

10.1 Weboldal – Gyakori problémák

A rendelés gomb inaktív marad

Ellenőrizze, hogy megadott-e szállítási címet a profilbeállításokban (/address). Ha még nem adott meg szállítási címet, az a rendelés leadásának előfeltétele.

Nem látok termékeket a termékek oldalon

Ellenőrizze az internetkapcsolatát. Ha az oldal betölt, de a termékek nem jelennek meg, töltse újra az oldalt (F5 billentyű). A probléma szerver oldali hiba esetén is előfordulhat; ilyen esetben próbálkozzon kicsivel később.

A profilkép nem frissül feltöltés után

A feltöltés után az oldal automatikusan frissíti a profilképet. Ha ez nem történik meg, töltse újra az oldalt. Ellenőrizze, hogy a feltöltött fájl JPG, JPEG vagy PNG formátumú-e.

A bejelentkezés nem működik a WPF alkalmazásban

A WPF alkalmazás az <https://notrox.hu> szerverhez csatlakozik. Ellenőrizze, hogy az internetkapcsolata aktív-e, és a szerver elérhető-e böngészőből. HTTP URL megadása esetén az alkalmazás ArgumentException-t dob és nem csatlakozik.

10.2 Kapcsolat és ügyfélszolgálat

Csatorna	Elérhetőség
E-mail	info@notrox.hu
Telefonszám	+36 20 111 1111
Postai cím	Budapest, Egressy út 71, 1149
Weboldal	https://notrox.hu/contact

11. Összefoglalás

Ez a felhasználói dokumentáció átfogó képet adott a NOTROX gaming platform mind a három kliensalkalmazásának telepítési és használati útmutatójáról. A web frontend, a WPF asztali alkalmazás és az Avalonia mobilalkalmazás együttesen egy komplex, egységes ökoszisztémát alkotnak, amelyen a vásárlók kényelmesen vásárolhatnak, az adminisztrátorok pedig hatékonyan kezelhetik a webáruház tartalmát.

Ha kérdése vagy problémája adódik bármely platform használatával kapcsolatban, forduljon bizalommal az ügyfélszolgálathoz a fent megadott elérhetőségeken.

Felhasznált irodalom és forrásmegjelölés

1. Bootstrap 5.3 dokumentáció. Elérhető: <https://getbootstrap.com/docs/5.3/> (Letöltve: 2025. március)
2. Avalonia UI dokumentáció. Elérhető: <https://docs.avaloniaui.net/> (Letöltve: 2025. március)
3. Microsoft WPF dokumentáció. Elérhető: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/> (Letöltve: 2025)
4. .NET 8 letöltési oldal. Elérhető: <https://dotnet.microsoft.com/en-us/download/dotnet/8.0> (Letöltve: 2025)
5. Font Awesome ikon könyvtár. Elérhető: <https://fontawesome.com/> (Letöltve: 2025)
6. Google Fonts. Elérhető: <https://fonts.google.com/> (Letöltve: 2025)

NOTROX

Gaming Webshop

FEJLESZTŐI DOKUMENTÁCIÓ

Szoftverfejlesztő vizsgaremek / Záródolgozat

Készítette: Péjő Kristóf, Molnár Márk Krisztián, Gróf Richárd László
Intézmény: Budapesti Műszaki SZC Egressy Gábor Két Tanítási Nyelvű Technikum

Weboldal: <https://notrox.hu>

Budapest, 2025–2026

TARTALOMJEGYZÉK

TARTALOMJEGYZÉK	38
1. Bevezetés.....	40
2. Specifikáció	41
2.1 A projekt bemutatása.....	41
2.2 Platformok és technológiák	41
2.3 Funkcionális követelmények – Web	41
2.4 Funkcionális követelmények – WPF asztali alkalmazás.....	41
2.5 Funkcionális követelmények – Avalonia mobilalkalmazás	42
2.6 Nem funkcionális követelmények	42
3. Rendszerarchitektúra	43
3.1 Magas szintű architektúra.....	43
3.2 Web projekt mappastruktúrája	43
3.3 Public mappa tartalma	43
3.4 WPF projekt mappastruktúrája.....	44
3.5 Avalonia projekt mappastruktúrája	44
4. Adatbázis dokumentáció	46
4.1 Adatbázis diagram.....	46
4.2 Táblák részletes leírása.....	46
4.2.1 Users tábla	46
4.2.2 Products tábla	47
4.2.3 Companies tábla	47
4.2.4 Orders tábla	47
4.2.5 OrderItems tábla.....	48
4.2.6 Addresses tábla.....	48
4.2.7 BillingAddresses tábla.....	48
4.2.8 Logs tábla	48
5. REST API dokumentáció	49
5.1 Általános elvek.....	49
5.2 Hitelesítési végpontok (Auth.js).....	49
5.2.1 Válaszkódok – Auth	49
5.3 Termék végpontok (Products.js)	50
5.4 Rendelési végpontok (Order.js).....	50
5.4.1 POST /createOrder – Request body és válaszkódok	50
5.5 Cím végpontok (Address.js / BillingAddress.js).....	50
5.6 Cég és Log végpontok.....	51

6. WPF asztali alkalmazás – részletes dokumentáció	51
6.1 Az alkalmazás célja és architektúrája.....	51
6.2 Osztálydiagram.....	51
6.3 MVVM rétegek	53
6.3.1 ViewModelBase.cs.....	53
6.3.2 ServerConnection.cs – HTTP réteg.....	53
6.3.3 LoginViewModel.cs	53
6.3.4 AdminViewModel.cs	53
6.3.5 ProductsViewModel.cs	54
6.4 OrdersClass.cs – TotalPrice számítás.....	54
6.5 Session.cs – munkamenet kezelés	54
7. Avalonia mobilalkalmazás – részletes dokumentáció.....	55
7.1 Az alkalmazás architektúrája	55
7.2 Platformspecifikus belépési pontok.....	55
7.3 MainViewModel és navigáció	55
7.4 Különbségek a WPF verziótól.....	55
8. Use Case diagram.....	56
8.1 Vásárló (User) funkciók	57
8.2 Adminisztrátor funkciók	57
9. Továbbfejlesztési lehetőségek.....	57
9.1 Rövid távú fejlesztések.....	57
9.2 Hosszú távú fejlesztések.....	58
10. Összefoglalás.....	58
Függelékek és mellékletek	59
I. melléklet – Képernyőképek	59
II. melléklet – package.json.....	61
Felhasznált irodalom és forrásmegjelölés	62

1. Bevezetés

A NOTROX egy modern, gaming perifériákra specializálódott webáruház-platform, amelynek fejlesztése szoftverfejlesztő vizsgaremekként valósult meg. A projekt három, egymással szorosan együttműködő alkalmazásból áll: egy webalapú frontend és Node.js backend, egy WPF asztali alkalmazás adminisztrációs célokra Windows platformon, és egy Avalonia UI mobilalkalmazás, amely Android eszközökön is futtatható.

A három platform ugyanazon REST API-t veszi igénybe, amelyet a Node.js + Express.js backend szolgáltat. Az adatok tárolása MySQL adatbázisban történik, amelyhez Sequelize ORM-en keresztül csatlakoznak az egyes szerveroldalas modulok. A dokumentáció célja, hogy egy fejlesztő számára teljes és egyértelmű képet adjon a rendszer felépítéséről, az egyes komponensek szerepéről és az API-végpontok viselkedéséről.

A weboldal URL-je: <https://notrox.hu>. A fejlesztés során a verziókövetés GitHub segítségével valósult meg. Az asztali (WPF) és mobilalkalmazás (Avalonia) szintén elérhető a projekt GitHub-repositoryjában.

2. Specifikáció

2.1 A projekt bemutatása

A NOTROX platform célja egy teljes körű gamer eszköz-értékesítési ökoszisztéma kiépítése, ahol a vásárlók böngészhetnek, rendelhetnek és nyomon követhetik megrendeléseiket egy reszponzív weboldalon, az adminisztrátorok pedig három különböző kliensalkalmazáson keresztül kezelhetik a termékkatalógust, a felhasználókat és a rendeléseket.

2.2 Platformok és technológiák

Platform	Technológia	Célközönség	Elérhetőség
Web frontend	HTML5, CSS3, JS, Bootstrap 5.3	Vásárlók	notrox.hu
Web backend	Node.js + Express.js + Sequelize	–	REST API
Adatbázis	MySQL	–	Szerver oldal
WPF Desktop	C# + WPF + MVVM + CommunityToolkit	Adminisztrátorok	Windows
Avalonia Mobile	C# + Avalonia UI + MVVM	Adminisztrátorok	Android /Desktop

2.3 Funkcionális követelmények – Web

- Felhasználói regisztráció és bejelentkezés (JWT autentikáció)
- Termékek listázása, szűrése és keresése
- Kosárkezelés (hozzáadás, mennyiségmódosítás, törlés) – localStorage alapú
- Rendelésfeladás bankártyaadatokkal és szállítási cím megadásával
- Rendeléstörténet megtekintése, PDF számla letöltése
- Profiladatok és profilkép kezelése (Multer feltöltés)
- Admin panel: felhasználó-, termék- és rendeléskezelés

2.4 Funkcionális követelmények – WPF asztali alkalmazás

- Admin bejelentkezés HTTPS-kapcsolaton keresztül
- Felhasználók listázása és törlése
- Termékek listázása, hozzáadása, szerkesztése és törlése

- Rendelések listázása és állapotmódosítása
- Szállítási és számlázási cím kezelése
- Keresés az egyes nézetekben (ISearchable interfész)
- Adatok exportálása CSV formátumban

2.5 Funkcionális követelmények – Avalonia mobilalkalmazás

- Azonos funkciók mint a WPF-ben, platformfüggetlen megvalósítással
- Android támogatás Avalonia UI segítségével
- Navigációkezelés MainViewModel + CurrentView pattern szerint
- MVVM architektúra, közös ServerConnection osztály

2.6 Nem funkcionális követelmények

Követelmény	Elvárás
Biztonság	HTTPS kötelező (ServerConnection konstruktor ellenőrzi), JWT Bearer token, bcrypt jelszóhashelés
Teljesítmény	API válaszidő < 500 ms normál terhelés mellett
Reszponzivitás	Bootstrap 5.3 breakpointok, mobil-kompatibilis UI
Karbantartathóság	MVVM architektúra, moduláris fájlszerkezet
Megbízhatóság	Hibakezelés try-catch minden HTTP hívásban, graceful fallback

3. Rendszerarchitektúra

3.1 Magas szintű architektúra

A rendszer háromrétegű architektúrán alapul: a kliensréteg (Web, WPF, Avalonia), a szerverréteg (Node.js + Express REST API), és az adatréteg (MySQL + Sequelize ORM). A kliensek mindegyike HTTP/HTTPS protokollon keresztül kommunikál a szerverrel, JSON formátumú adatcserével.

3.2 Web projekt mappastruktúrája

A web projekt gyökérkönyvtára tartalmazza a Node.js backend fájljait. A public/ almappa az összes statikus frontend erőforrást foglalja magában.

Fájl / Mappa	Leírás
server.js	Express szerver belépési pontja, middleware-ek és route regisztráció
Auth.js	Regisztráció, bejelentkezés, jelszócsere végpontok
User.js	Felhasználói CRUD, profilkép feltöltés (Multer)
Products.js	Termékkatalógus CRUD végpontok
Order.js	Rendelés létrehozás, lekérdezés, állapotkezelés
Company.js	Gyártók lekérdezése
Address.js	Szállítási cím kezelése
BillingAddress.js	Számlázási cím kezelése
Log.js	Naplóbejegyzések kezelése
dbhandler.js	Sequelize konfiguráció és modell exportok
public/	Frontend: HTML oldalak, CSS stílusok, JS scriptek
uploads/	Multer által feltöltött profilképek

3.3 Public mappa tartalma

HTML fájl	URL (clean)	Hozzáférés
index.html	/	Nyilvános
login.html	/login	Vendég (kijelentkezett)
register.html	/register	Vendég (kijelentkezett)

HTML fájl	URL (clean)	Hozzáférés
products.html / profil.html	/products, /profil	Nyilvános / Bejelentkezett
cart.html	/cart	Bejelentkezett felhasználó
recentOrders.html	/recentorders	Bejelentkezett felhasználó
success.html	/success	Bejelentkezett (sikeres rendelés)
admin.html	/admin	Admin bejelentkezési oldal
adminpanel.html / product-add.html	/adminpanel, /product-add	Admin jogkör
product-list.html	/product-list	Admin jogkör
order-list.html	/order-list	Admin jogkör
404.html	/404	Nyilvános

3.4 WPF projekt mappastruktúrája

Mappa / Fájl	Leírás
Model/	Adatmodellek: UsersClass, UserAddressesClass, ProductsClass, OrdersClass, OrderItemClass, AddressClass, BillingAddressClass, CompanyClass, Message.cs.
View/	XAML nézetek: MainWindow, AdminWindow, AddProductWindow, EditProductsWindow, EditAddressesWindow, Orders, Products, Users.
ViewModel/	ViewModelek: LoginViewModel, AdminViewModel, ProductsViewModel, UsersViewModel, AddProductViewModel, EditAddressesViewModel, EditProductsViewModel, OrdersViewModel, ViewModelBase.
Services/ServerConnection.cs	HTTP kliens, JWT kezelés, összes API hívás implementációja
Services/Session.cs	Statikus Session osztály a bejelentkezett felhasználónév tárolásához
Interfaces/ISearchable.cs	Keresési funkciót definiáló interfész
Notrox_WPF_Test/Test1.cs	MSTest unit tesztek a ServerConnection és OrdersClass osztályokhoz

3.5 Avalonia projekt mappastruktúrája

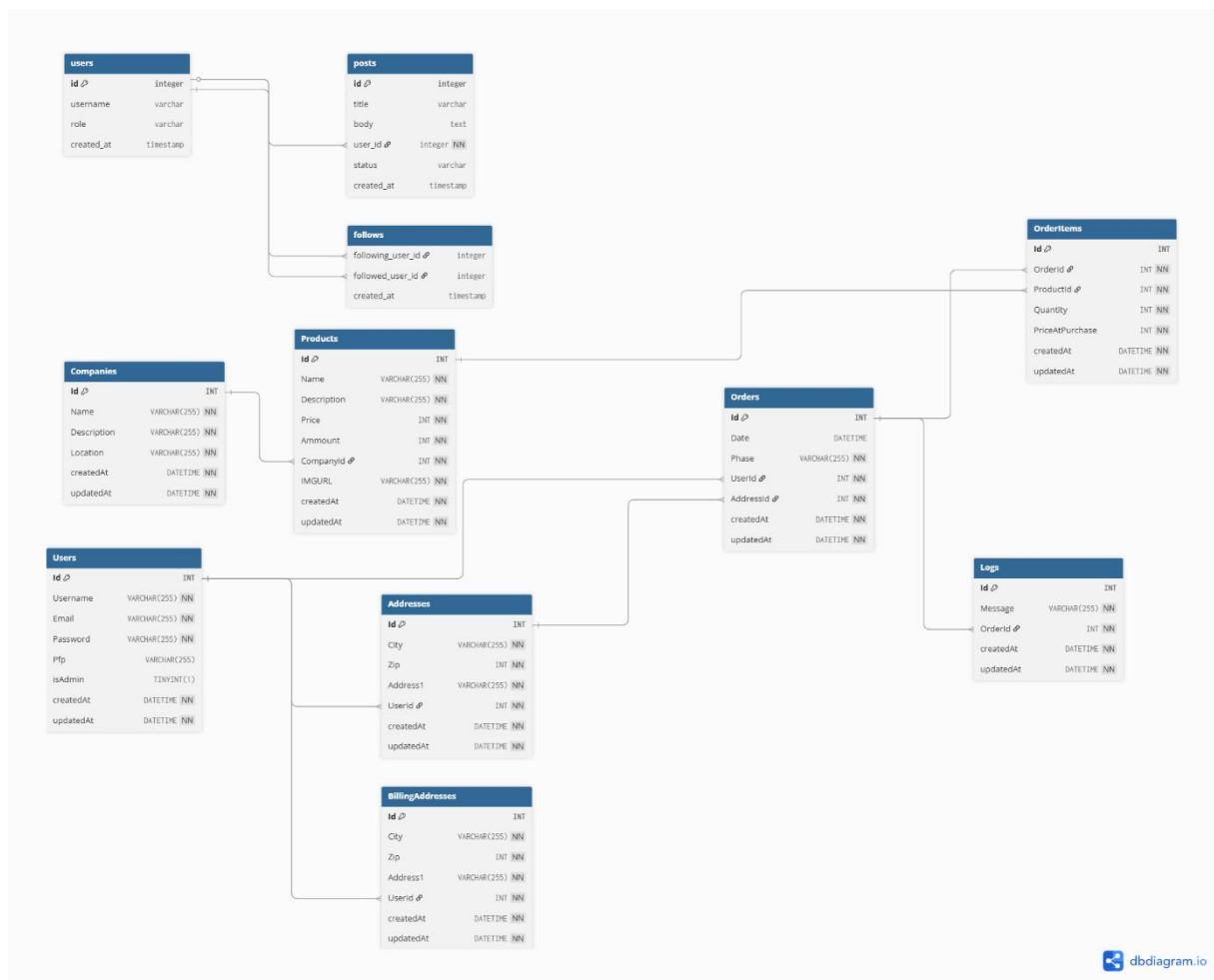
Mappa / Fájl	Leírás
Models/	Azonos adatmodellek mint WPF-ben (UsersClass, UserAddressesClass, ProductsClass, OrdersClass, OrderItemClass, AddressClass, BillingAddressClass, CompanyClass)
Views/	Avalonia AXAML nézetek: AdminView, AddProductView, EditProductsView, EditAddressesView, Orders, Products, Users, MainView, StartupView
ViewModels/	MVVM ViewModelek: AddProductViewModel, AdminViewModel, EditAddressesViewModel, EditProductsViewModel,

Mappa / Fájl	Leírás
	MainViewModel, OrdersViewModel, ProductsViewModel, StartupViewModel, UsersViewModel, ViewModelBase
Services/ServerConnection.cs	Azonos logikájú HTTP kliens mint WPF-ben, Avalonia-kompatibilis
Services/Session.cs	Statikus Session osztály
Notrox.Android/	Android-specifikus belépési pont
Notrox_Test/Test1.cs	MSTest unit tesztek (azonos struktúra mint WPF tesztek)

4. Adatbázis dokumentáció

4.1 Adatbázis diagram

Az adatbázis MySQL relációs adatbázis-kezelőrendszert alkalmaz. Az alábbi diagram a dbdiagram.io segítségével készült, és az összes táblát, mezőt és kapcsolatot szemléltetővé teszi.



4.2 Táblák részletes leírása

4.2.1 Users tábla

Mező	Típus	Leírás	Megkötés
Id	INT	Elsődleges kulcs	AUTO INCREMENT, PK
Username	VARCHAR(255)	Felhasználónév	NOT NULL
Email	VARCHAR(255)	E-mail cím	UNIQUE, NOT NULL

Mező	Típus	Leírás	Megkötés
Password	VARCHAR(255)	Bcrypt hashelt jelszó	NOT NULL
Pfp	VARCHAR(255)	Profilkép elérési út	NULLABLE
isAdmin	TINYINT(1)	Admin jogkör jelzője	DEFAULT: 0
createdAt / updatedAt	DATETIME	Sequelize automatikus időbélyeg	NOT NULL

4.2.2 Products tábla

Mező	Típus	Leírás	Megkötés
Id	INT	Elsődleges kulcs	AUTO INCREMENT
Name	VARCHAR(255)	Termék neve	NOT NULL
Description	VARCHAR(255)	Termék leírása	NOT NULL
Price	INT	Ár forintban	NOT NULL
Ammount	INT	Raktárkészlet darabban	NOT NULL
CompanyId	INT	Idegen kulcs → Companies	FK, NOT NULL
IMGURL	VARCHAR(255)	Termékkép URL-je	NOT NULL

4.2.3 Companies tábla

Mező	Típus	Leírás	Megkötés
Id	INT	Elsődleges kulcs	AUTO INCREMENT
Name	VARCHAR(255)	Gyártó neve (pl. Logitech)	NOT NULL
Description	VARCHAR(255)	Gyártó leírása	NULLABLE
Location	VARCHAR(255)	Gyártó helyszíne	NULLABLE

4.2.4 Orders tábla

Mező	Típus	Leírás	Megkötés
Id	INT	Elsődleges kulcs	AUTO INCREMENT
Date	DATETIME	Rendelés időpontja	NOT NULL
Phase	VARCHAR(255)	Rendelés állapota	NOT NULL
UserId	INT	Rendelő felhasználó	FK → Users
AddressId	INT	Szállítási cím ID-ja	FK → Addresses

4.2.5 OrderItems tábla

Mező	Típus	Leírás	Megkötés
Id	INT	Elsődleges kulcs	AUTO INCREMENT
OrderId	INT	Rendelés azonosítója	FK → Orders
ProductId	INT	Termék azonosítója	FK → Products
Quantity	INT	Rendelt mennyiség	NOT NULL
PriceAtPurchase	INT	Vételkori ár	NOT NULL

4.2.6 Addresses tábla

Mező	Típus	Leírás	Megkötés
Id	INT	Elsődleges kulcs	AUTO INCREMENT
City	VARCHAR(255)	Város neve	NOT NULL
Zip	INT	Írányítószám	NOT NULL
Address1	VARCHAR(255)	Utca, házszám	NOT NULL
UserId	INT	Felhasználó azonosítója	FK → Users

4.2.7 BillingAddresses tábla

A BillingAddresses tábla megegyezik az Addresses tábla szerkezetével, különálló entitásként kezelve a számlázási és szállítási adatokat. Mezői: Id, City, Zip, Address1, UserId (FK → Users), createdAt, updatedAt.

4.2.8 Logs tábla

Mező	Típus	Leírás	Megkötés
Id	INT	Elsődleges kulcs	AUTO INCREMENT
Message	VARCHAR(255)	Naplóüzenet szövege	NOT NULL
OrderId	INT	Kapcsolt rendelés	FK → Orders

5. REST API dokumentáció

5.1 Általános elvek

A backend RESTful API-t valósít meg. Minden kérés JSON formátumban kommunikál. A védett végpontok Bearer JWT token igényelnek az Authorization fejlécben: Authorization: Bearer <token>. Az admin végpontokhoz a tokennek isAdmin: true értéket kell tartalmaznia. A szerver alapértelmezett portja 3000, éles üzemeltetésnél reverse proxy (pl. Nginx) kezeli a HTTPS-t.

5.2 Hitelesítési végpontok (Auth.js)

Metódus	URL	Leírás	Auth	Body
POST	/UserLogin	Bejelentkezés – JWT token generálás	–	{ Username, Password }
POST	/UserRegister	Regisztráció – új felhasználó	–	{ Username, Email, Password }
PUT	/EditUser	Adatmódosítás (név, email, jelszó)	Igen	{ Username?, Email?, Password?, OldPassword? }
DELETE	/deleteUser/:id	Felhasználó törlése	Admin	–
GET	/getUser	Saját adatok lekérdezése	Igen	–
GET	/getAllUsers	Összes felhasználó (admin)	Admin	–

5.2.1 Válaszkódok – Auth

Végpont	Kód	Szituáció
POST /UserLogin	200 OK	Sikeres bejelentkezés, visszatér: { token }
POST /UserLogin	401 Unauthorized	Helytelen jelszó
POST /UserLogin	404 Not Found	Nem létező felhasználónév
POST /UserRegister	201 Created	Sikeres regisztráció – { message: 'Sikeres regisztráció' }
POST /UserRegister	409 Conflict	Már létező felhasználó – { message: 'Van már ilyen felhasználó' }
PUT /EditUser	200 OK	Sikeres módosítás – { message: 'Sikeres adatmódosítás!' }
PUT /EditUser	400 Bad Request	Nincs módosítandó adat – { message: 'Nincs módosítandó adat.' }

5.3 Termék végpontok (Products.js)

Metódus	URL	Leírás	Auth
GET	/getproduct	Összes termék + Company join	–
POST	/postproduct	Új termék létrehozása	Admin
PUT	/updateproduct/:id	Termék adatainak módosítása	Admin
DELETE	/deleteproduct/:id	Termék törlése	Admin

5.4 Rendelési végpontok (Order.js)

Metódus	URL	Leírás	Auth
POST	/createOrder	Rendelés feladása – készlet csökkentés	Igen
GET	/getMyOrders	Saját rendelések (OrderItems + Address)	Igen
GET	/getAllOrders	Összes rendelés (admin)	Admin
PUT	/updateOrderPhase/:id	Rendelési állapot módosítása	Admin

5.4.1 POST /createOrder – Request body és válaszkódok

Mező	Típus	Kötelező	Leírás
AddressId	integer	Igen	A szállítási cím azonosítója
cart	array	Igen	Tételek: [{Id, Name, Price, quantity}]

HTTP kód	Szituáció	Válasz üzenet
201 Created	Sikeres rendelés	{ message: 'Rendelés sikeres!', orderId: N }
400 Bad Request	Üres kosár	{ message: 'Üres a kosár' }
400 Bad Request	Hiányzó AddressId	{ message: 'Hiányzó cím azonosító' }

5.5 Cím végpontok (Address.js / BillingAddress.js)

Metódus	URL	Leírás	Auth
POST	/AddressRegister	Szállítási cím regisztrálása	Igen
GET	/AddressGets	Saját szállítási cím lekérdezése	Igen
PUT	/EditAddress	Szállítási cím módosítása	Igen
GET	/getUserAddresses/:id	Adott user cím adatai (WPF admin)	Admin
POST	/BillingAddressRegister	Számlázási cím regisztrálása	Igen
GET	/BillingAddressGets	Saját számlázási cím	Igen

Metódus	URL	Leírás	Auth
PUT	/EditBillingAddress	Számlázási cím módosítása	Igen

5.6 Cég végpontok

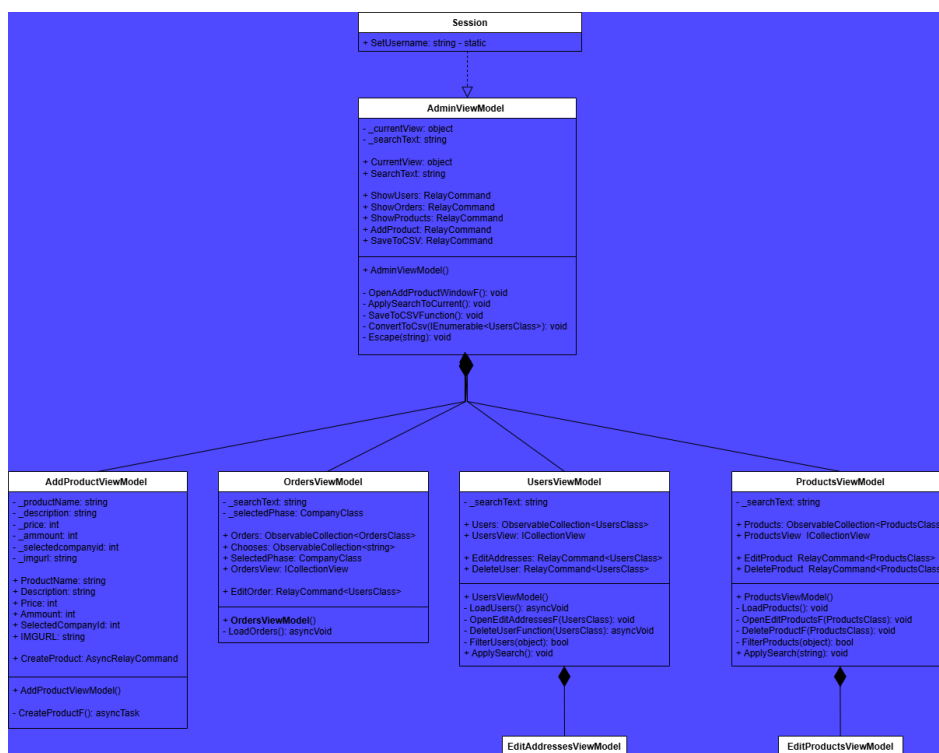
Metódus	URL	Leírás	Auth
GET	/getcompany	Összes gyártó listája	–

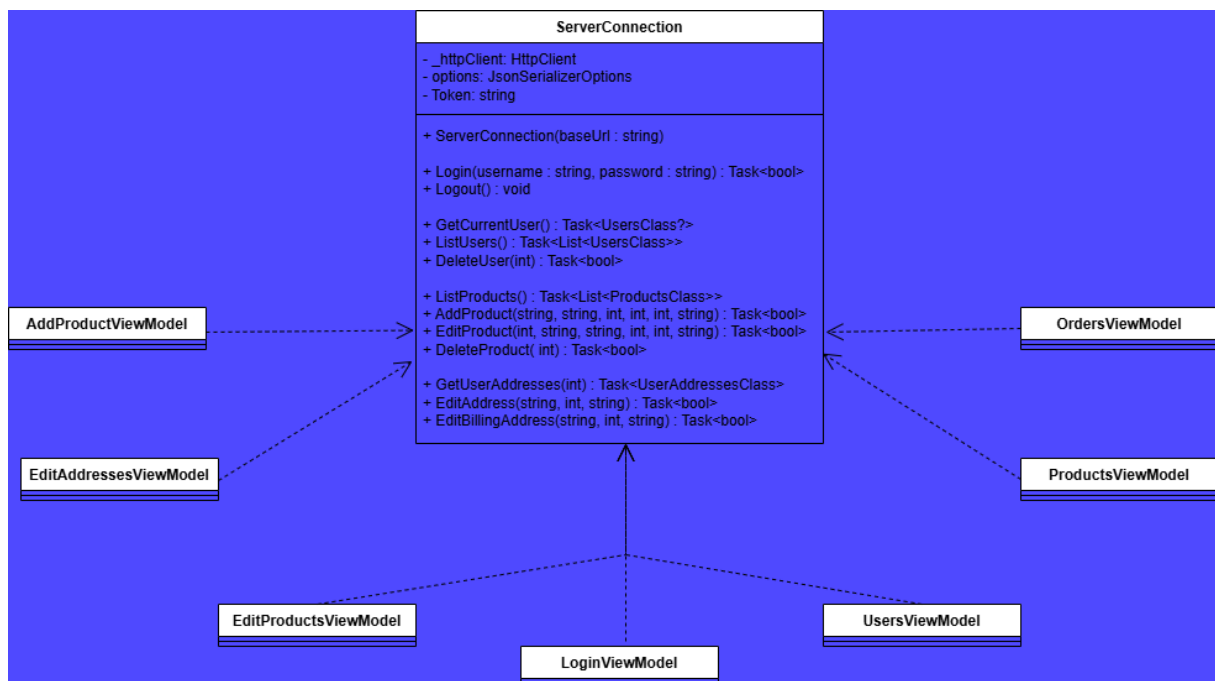
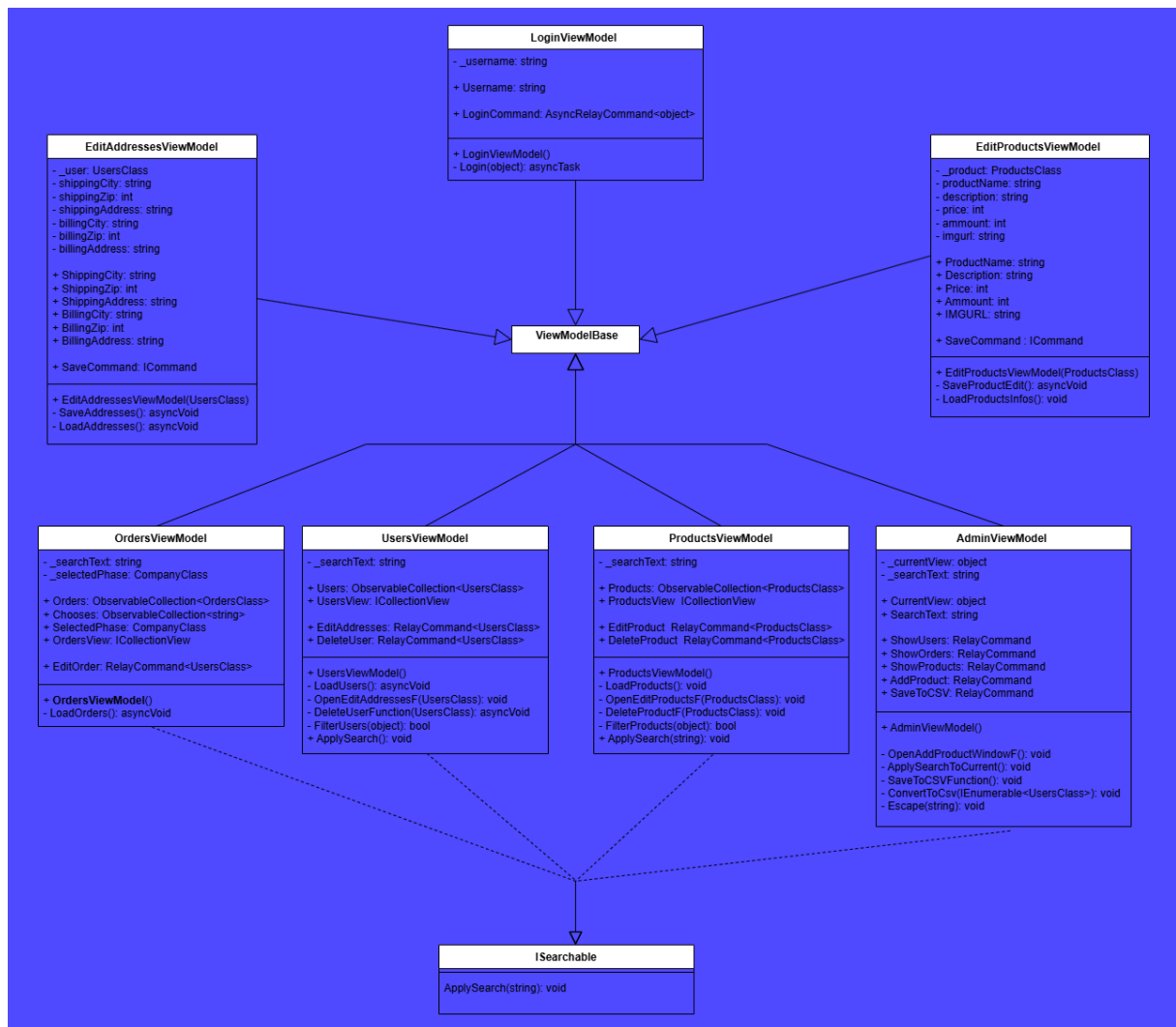
6. WPF asztali alkalmazás – részletes dokumentáció

6.1 Az alkalmazás célja és architektúrája

A WPF alkalmazás az adminisztrátorok számára készült Windows asztali kliensalkalmazás. Az MVVM (Model-View-ViewModel) architektúrát követi, a CommunityToolkit.Mvvm könyvtár segítségével. Az alkalmazás az összes adatot a közös NOTROX REST API-n keresztül olvassa és módosítja.

6.2 Osztálydiagram





6.3 MVVM rétegek

6.3.1 *ViewModelBase.cs*

Az összes ViewModel közös őssztálya, implementálja az INotifyPropertyChanged interfészt. A CallerMemberName attribútum automatikusan biztosítja a property nevét a változásértesítőhöz.

```
public abstract class ViewModelBase : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;
    protected virtual void OnPropertyChanged([CallerMemberName] String?
propertyName = null)
    {
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
    }
}
```

6.3.2 *ServerConnection.cs – HTTP réteg*

A ServerConnection osztály az összes HTTP kommunikációt kezeli a System.Net.Http.HttpClient segítségével. Konstruktorában ellenőrzi, hogy az URL HTTPS protokollt használ-e; HTTP esetén ArgumentException-t dob. A bejelentkezéskor kapott JWT tokenet automatikusan elmenti és minden következő kérés fejlécébe illeszti.

```
public ServerConnection(string baseUrl)
{
    if (!baseUrl.StartsWith("https://"))
        throw new ArgumentException("Insecure connection not allowed");
    _httpClient = new HttpClient();
    _httpClient.BaseAddress = new Uri(baseUrl);
}
```

6.3.3 *LoginViewModel.cs*

A bejelentkezési nézet ViewModelje az AsyncRelayCommand pattern-t használja a CommunityToolkit.Mvvm csomagból. Az implementáció a PasswordBox-ot paraméterként kapja (WPF biztonsági megfontolásból a jelszó nem köthető közvetlenül ViewModel propertyre).

```
public AsyncRelayCommand<object> LoginCommand { get; }
private async Task Login(object param)
{
    if (param is not PasswordBox passwordBox) return;
    bool success = await App.Server.Login(Username, passwordBox.Password);
    if (!success) { MessageBox.Show("Hibás felhasználónév vagy jelszó.");
return; }
    Session.SetUsername = user.Username;
    new AdminWindow().Show();
}
```

6.3.4 *AdminViewModel.cs*

Az AdminViewModel kezeli a fő admin ablak navigációját. A CurrentView property tartalmazza az éppen aktív ViewModel-t. A keresés globálisan működik az ISearchable interfészen keresztül.

```
public RelayCommand ShowUsers { get; }
public RelayCommand ShowOrders { get; }
public RelayCommand ShowProducts { get; }
public string SearchText { get => _searchText; set {
    _searchText = value; OnPropertyChanged();
    if (CurrentView is ISearchable s) s.ApplySearch(_searchText); }}
```

6.3.5 ProductsViewModel.cs

A termékek listázásához ObservableCollection<ProductsClass>-t és ICollectionView szűrést alkalmaz. A szűrés valós idejű: a FilterProducts metódus ellenőrzi, hogy a termék neve vagy gyártója tartalmazza-e a keresési kifejezést.

6.4 OrdersClass.cs – TotalPrice számítás

Az OrdersClass egy speciális computed property-t tartalmaz: a TotalPrice a rendelési tételek alapján automatikusan kiszámítja a rendelés végösszegét. Ez a property a unit tesztek fő célpontja.

```
public decimal TotalPrice =>
    Items?.Sum(i => i.PriceAtPurchase * i.Quantity) ?? 0;
```

6.5 Session.cs – munkamenet kezelés

A Session statikus osztály egyszerű, alkalmazásszintű állapot-tárolást biztosít. A SetUsername property tárolja a bejelentkezett admin felhasználónevét, amelyet az admin ablak fejlécében jelenít meg az alkalmazás.

```
public static class Session
{
    public static string SetUsername { get; set; } = "";
}
```

7. Avalonia mobilalkalmazás – részletes dokumentáció

7.1 Az alkalmazás architektúrája

Az Avalonia alkalmazás a WPF-hez hasonló MVVM architektúrát alkalmaz, amely lehetővé teszi a mobilos megjelenítést. Az Avalonia UI keretrendszer lehetővé teszi, hogy a C# és AXAML kódok Android platformon működjenek. A navigáció a MainViewModel által kezelt CurrentView pattern-t alkalmaz, ahol a StartUpView jelenti a kiindulópontot.

7.2 Platformspecifikus belépési pontok

Platform	Projekt	Belépési pont
Android	Notrox.Android	MainActivity.cs → BuildAvaloniaApp()

7.3 MainViewModel és navigáció

A MainViewModel egy CurrentView property-t tart karban, amelyre az AXAML-ben Data Template keresőn (ViewLocator) keresztül renderelődnek a nézetek. Az inicializálás során a StartUpViewModel töltődik be, amely a bejelentkezési képernyőt mutatja.

```
public class MainViewModel : ViewModelBase
{
    private ViewModelBase _currentView;
    public ViewModelBase CurrentView {
        get => _currentView;
        set { _currentView = value; OnPropertyChanged(); }
    }
    public MainViewModel() {
        CurrentView = new StartUpViewModel(this);
    }
}
```

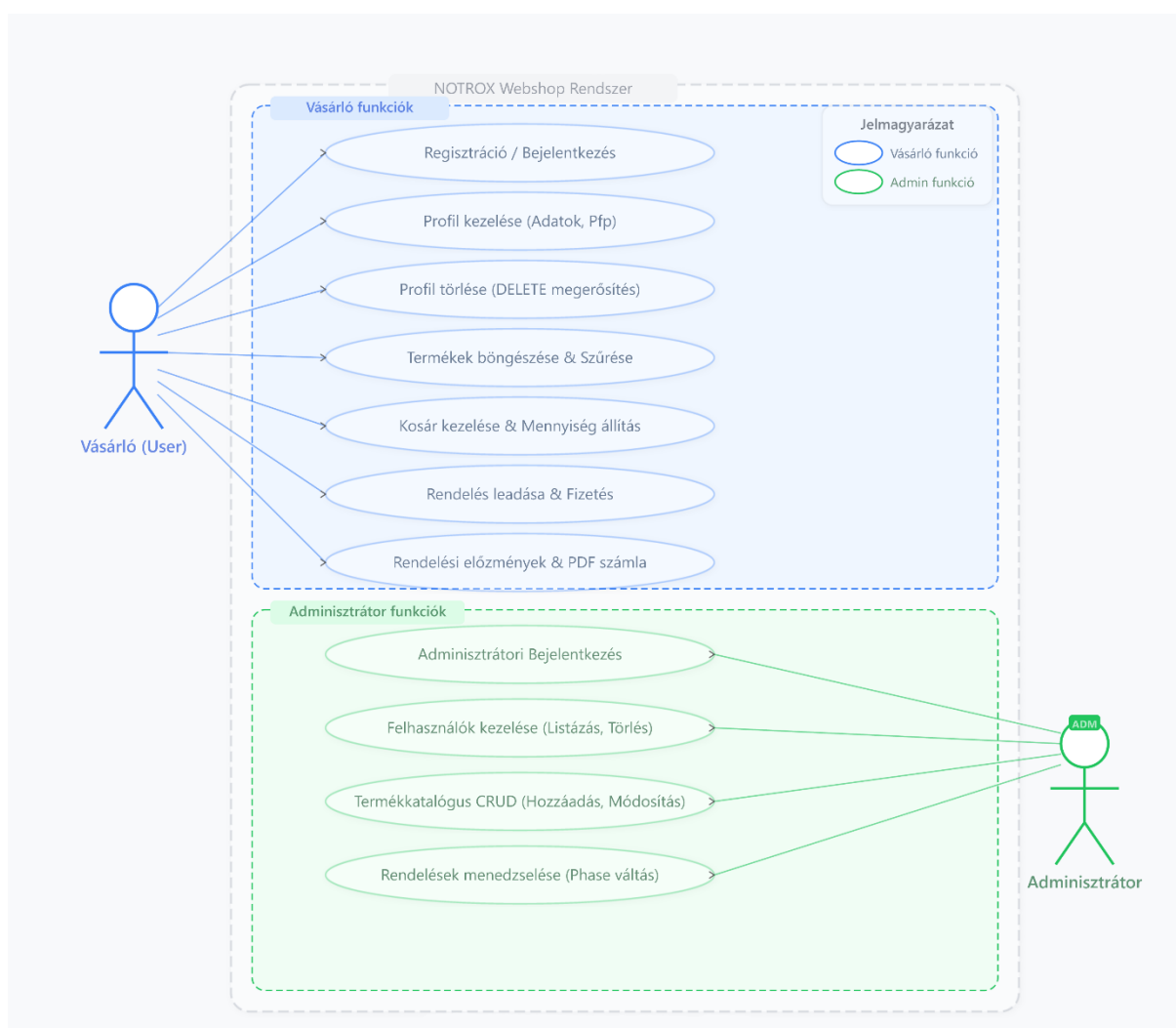
7.4 Különbségek a WPF verziótól

Jellemző	WPF	Avalonia
UI Framework	Windows Presentation Foundation	Avalonia UI (cross-platform)
Nézet kiterjesztés	.xaml	.axaml
Ablakkezelés	Window.ShowDialog(), Close()	CurrentView csere MainViewModel-ben
Platformtámogatás	Csak Windows	Android

Jellemző	WPF	Avalonia
PasswordBox	WPF natív, AsyncRelayCommand<object>	Avalonia TextBox (PasswordChar mód)
Assert szintaxis (teszt)	Assert.ThrowsException<T>()	Assert.Throws<T>()

8. Use Case diagram

Az alábbi use case diagram a NOTROX rendszer két fő szereplőjének (Vásárló és Adminisztrátor) funkcióit mutatja be egységes, rendszerszintű nézetben.



8.1 Vásárló (User) funkciók

- Regisztráció / Bejelentkezés – JWT alapú azonosítás
- Profil kezelése – adatok és profilkép (Pfp) módosítása
- Profil törlése – DELETE megerősítéssel
- Termékek böngészése és szűrése
- Kosár kezelése és mennyiség állítása
- Rendelés leadása és fizetés
- Rendelési előzmények és PDF számla letöltése

8.2 Adminisztrátor funkciók

- Adminisztrátori bejelentkezés
- Felhasználók kezelése – listázás és törlés
- Termékkatalógus CRUD – hozzáadás, szerkesztés, törlés
- Rendelések menedzselése – Phase-váltás (pl. Feldolgozás → Teljesítve)

9. További fejlesztési lehetőségek

9.1 Rövid távú fejlesztések

- E-mail értesítések: rendelés visszaigazolása és állapotfrissítés Nodemailer segítségével
- Termékkategória szűrő: legördülő szűrő a terméklistán
- Jelszó visszaállítás: e-mail alapú tokenes megoldás
- Termék értékelések: csillagos értékelési rendszer
- Keresési automatikus kiegészítés (autocomplete) a termékkeresőbe

9.2 Hosszú távú fejlesztések

- React vagy Next.js frontend: az oldalak átírása modern SPA keretrendszerbe
- Valós fizetési integráció: Stripe vagy Barion payment gateway
- Termékképek feltöltése URL helyett közvetlen fájlként
- Kupon és akciókódszisztem
- Analitikai dashboard – értékesítési adatok vizualizálása az admin felületen
- Push értesítések a mobilalkalmazásban rendelési állapotváltáskor
- Automata tesztelési lefedettség növelése: Playwright E2E tesztek hozzáadása

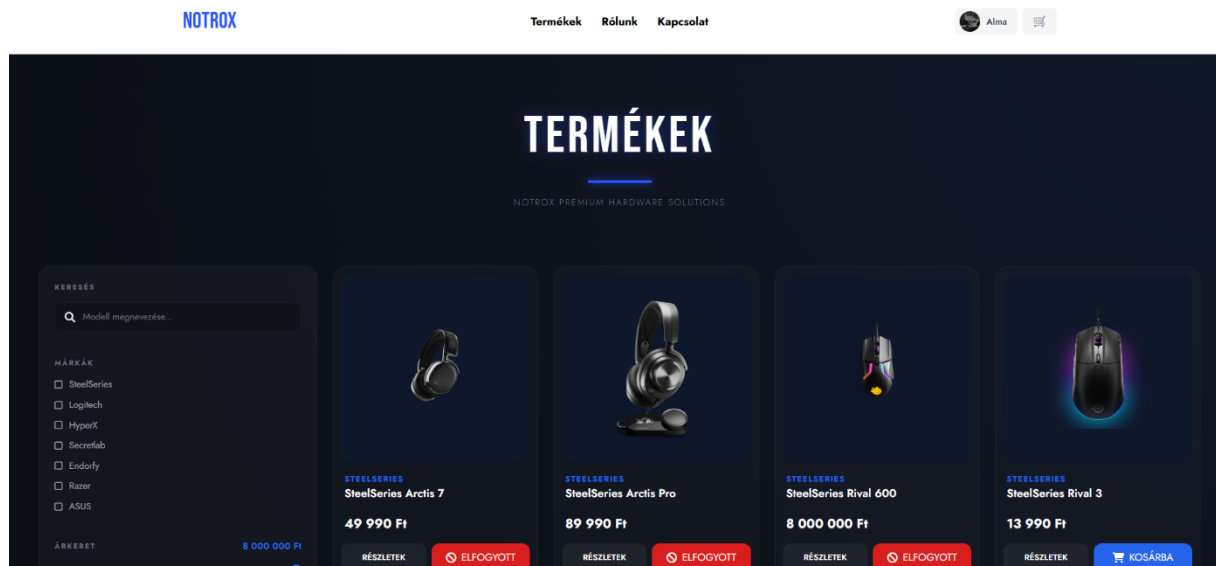
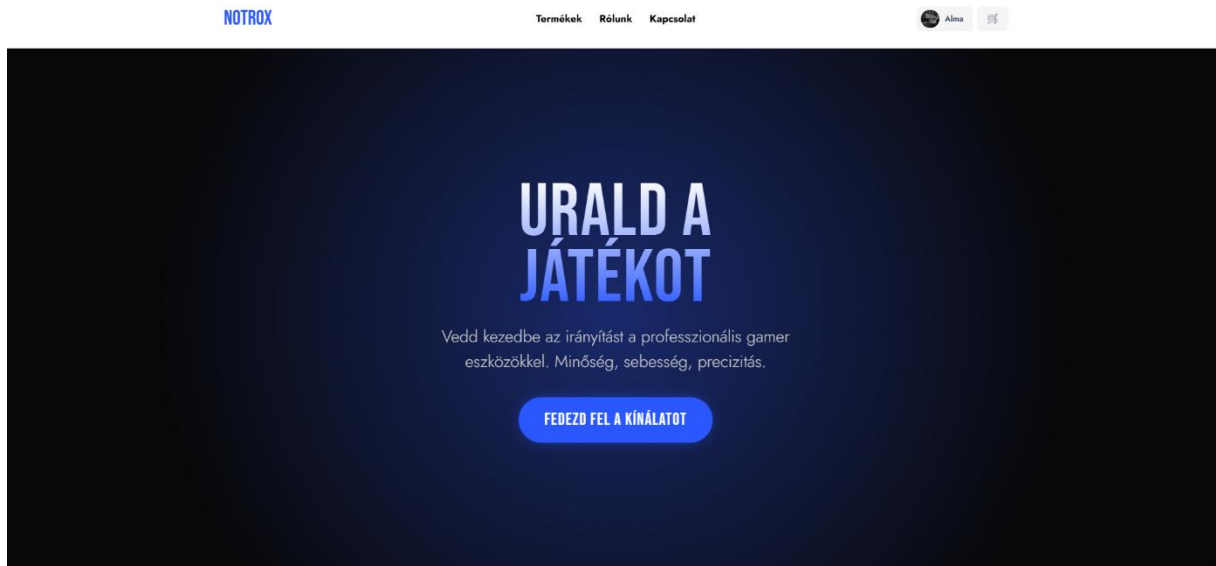
10. Összefoglalás

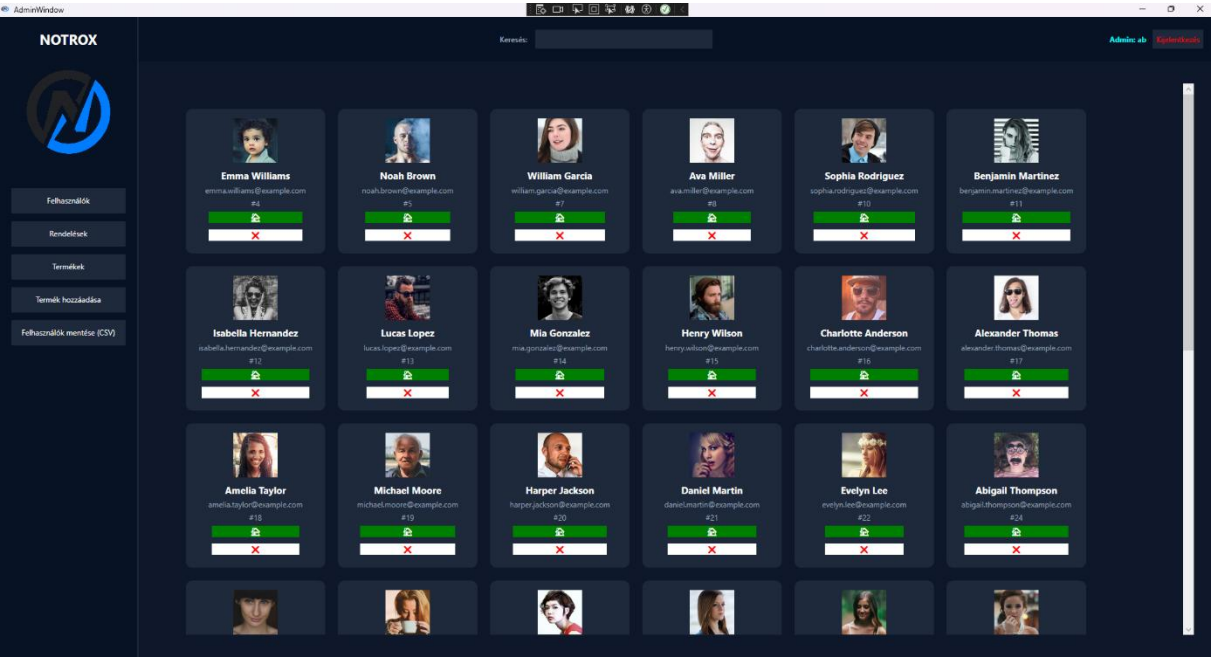
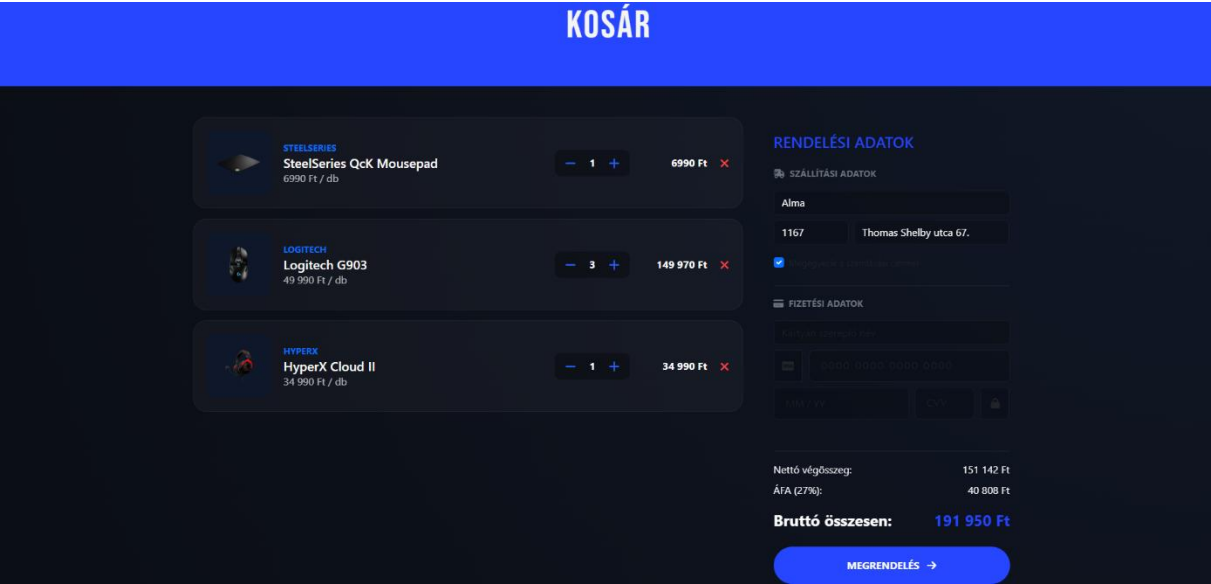
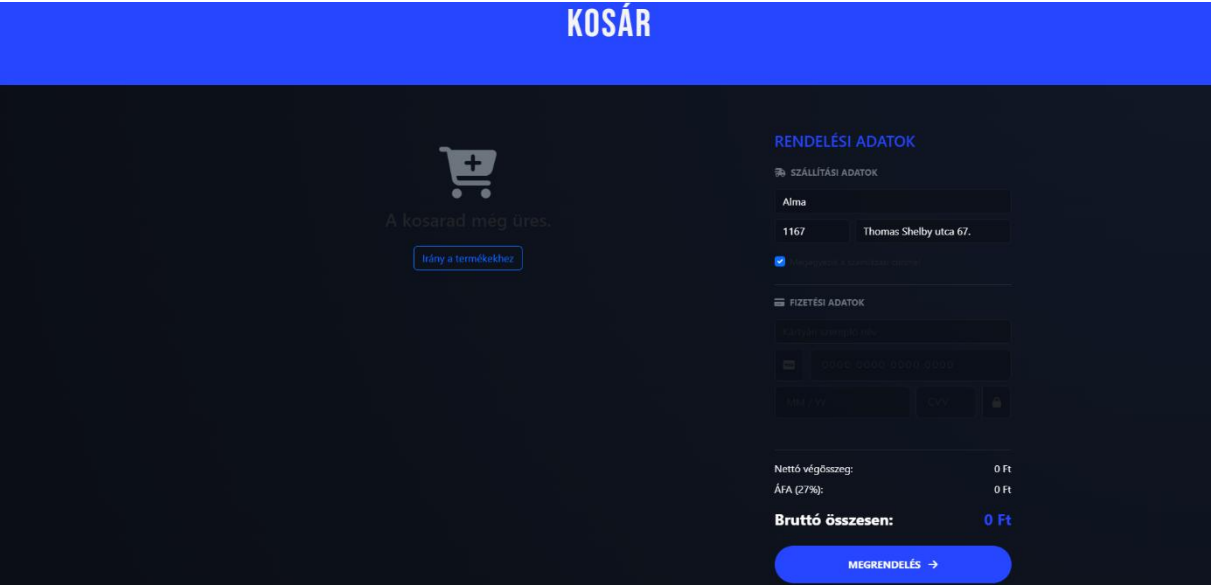
A NOTROX gaming platform egy komplex, három kliensplatformból álló szoftverrendszer, amely a modern webfejlesztési és asztali fejlesztési technológiák széleskörű alkalmazását demonstrálja. A projekt fejlesztése során egy valós üzleti igényeket kielégítő webáruház és hozzá kapcsolódó adminisztrációs kliensek kerültek megvalósításra.

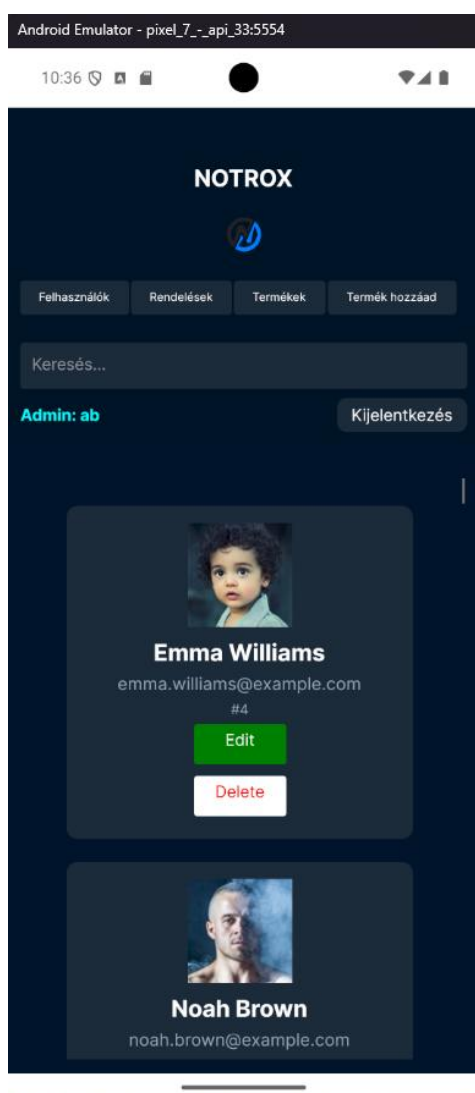
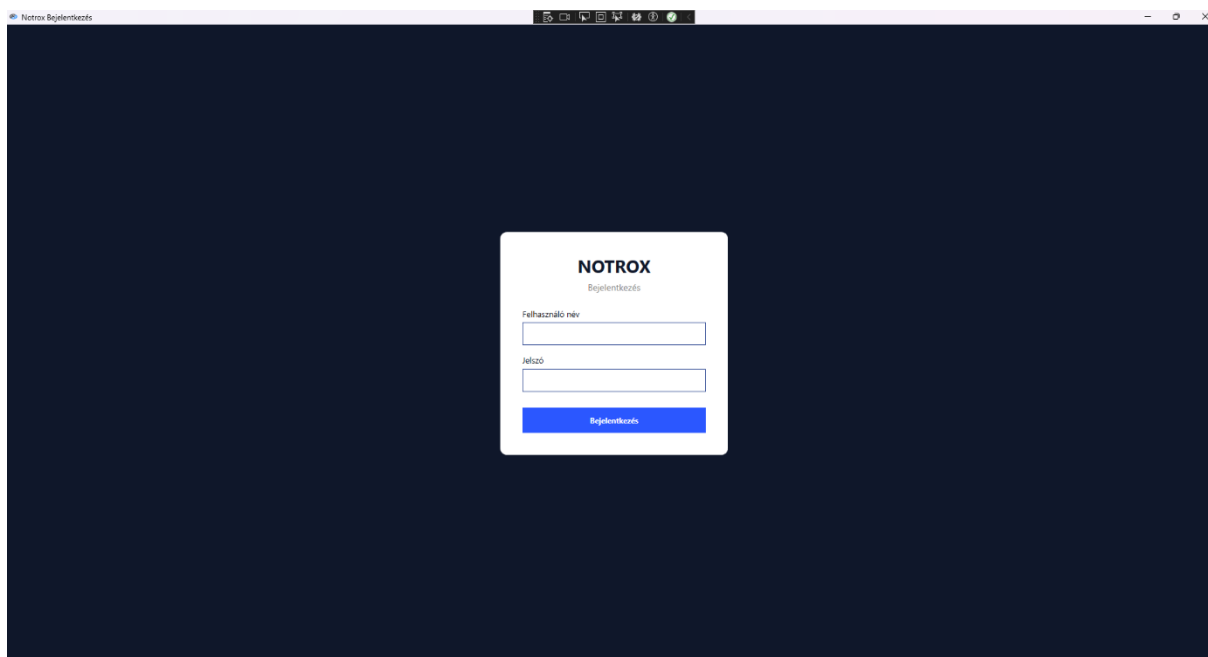
A fejlesztés főbb eredményei: REST API tervezés és implementáció Node.js + Express.js + Sequelize technológiával; JWT-alapú hitelesítési rendszer; WPF MVVM alkalmazás CommunityToolkit.Mvvm segítségével; Avalonia UI alkalmazás; reszponzív Bootstrap 5.3 alapú web frontend; automatizált tesztelési infrastruktúra (MSTest + Jest + Supertest). A projekt kielégíti az összes definiált funkcionális és nem funkcionális követelményt. A kódrendszer karbantartható, moduláris szerkezetű, és stabil alapot nyújt a jövőbeli bővítések számára.

Függelékek és mellékletek

I. melléklet – Képernyőképek







II. melléklet – package.json

```
{
  "dependencies": {
    "bcrypt": "^6.0.0",
    "cors": "^2.8.6",
    "dotenv": "^17.2.4",
    "express": "^5.2.1",
    "jest": "^30.2.0",
    "jsonwebtoken": "^9.0.3",
    "multer": "^2.0.2",
    "mysql2": "^3.16.3",
    "sequelize": "^6.37.7",
    "supertest": "^7.2.2"
  }
}
```

Felhasznált irodalom és forrásmegjelölés

-
1. Express.js dokumentáció. Elérhető: <https://expressjs.com/> (Letöltve: 2025. március)
 2. Sequelize ORM v6 dokumentáció. Elérhető: <https://sequelize.org/docs/v6/> (Letöltve: 2025. március)
 3. JWT.io – JSON Web Tokens bevezető. Elérhető: <https://jwt.io/introduction> (Letöltve: 2025. február)
 4. Bootstrap 5.3 dokumentáció. Elérhető: <https://getbootstrap.com/docs/5.3/> (Letöltve: 2025. február)
 5. CommunityToolkit.Mvvm dokumentáció. Elérhető: <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/> (Letöltve: 2025. március)
 6. Avalonia UI dokumentáció. Elérhető: <https://docs.avaloniaui.net/> (Letöltve: 2025. március)
 7. MDN Web Docs – Web API referencia. Elérhető: <https://developer.mozilla.org/> (Letöltve: 2025)
 8. bcryptjs npm dokumentáció. Elérhető: <https://www.npmjs.com/package/bcryptjs> (Letöltve: 2025. március)
 9. Multer fájlfeltöltő middleware. Elérhető: <https://github.com/expressjs/multer> (Letöltve: 2025. március)

10. Microsoft WPF dokumentáció. Elérhető: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/> (Letöltve: 2025)

11. dbdiagram.io – Adatbázis diagram eszköz. Elérhető: <https://dbdiagram.io/> (Letöltve: 2025)

NOTROX

Gaming Webshop

TESZTELÉSI DOKUMENTÁCIÓ

Szoftverfejlesztő vizsgaremek / Záródolgozat

Készítette: Péjő Kristóf, Molnár Márk Krisztián, Gróf Richárd László
Intézmény: Budapesti Műszaki SZC Egressy Gábor Két Tanítási Nyelvű Technikum

Weboldal: <https://notrox.hu>

Budapest, 2025–2026

TARTALOMJEGYZÉK

TARTALOMJEGYZÉK	65
1. Bevezetés a tesztelési dokumentációba	66
2. Tesztelési stratégia	67
2.1 Alkalmazott tesztelési típusok.....	67
2.2 Tesztkörnyezet	67
3. Web – Manuális tesztesetek	68
3.1 Regisztráció – manuális tesztek	68
3.2 Bejelentkezés – manuális tesztek	68
3.3 Jogosultságvédelem – manuális tesztek	69
3.4 Kosár – manuális tesztek.....	69
3.5 Rendelési folyamat – manuális tesztek	70
3.6 Profil és cím kezelés – manuális tesztek	71
3.7 Reszponzivitás – manuális tesztek	72
4. Backend – Automata API tesztek (Jest + Supertest).....	73
4.1 Felhasználói API tesztek	73
4.1.1 POST /UserLogin	73
4.1.2 POST /UserRegister	73
4.1.3 PUT /EditUser	74
4.2 Cím API tesztek (Address).....	74
4.3 Számlázási cím API tesztek (BillingAddress)	75
4.4 Rendelés API tesztek.....	75
5. WPF – Unit tesztek (MSTest)	76
5.1 OrdersClass.TotalPrice unit tesztek	76
5.2 ServerConnection – biztonsági és HTTP unit tesztek	77
6. Avalonia – Unit tesztek (MSTest).....	79
6.1 OrdersClass.TotalPrice tesztek – Avalonia	79
6.2 ServerConnection tesztek – Avalonia	80
7. Tesztelési összefoglaló és statisztikák.....	81
7.1 Összes teszteset összesítője	81
7.2 Azonosított és javított hibák.....	81
7.3 Következtetések	81
8. Összefoglalás.....	82
Felhasznált irodalom és forrásmegjelölés	82

1. Bevezetés a tesztelési dokumentációba

Ez a dokumentum a NOTROX gaming platform összes szoftverkomponensének tesztelési tevékenységét foglalja össze. A tesztelés három rendszerre terjed ki: a webes frontend és Node.js backend, a WPF asztali alkalmazás, valamint az Avalonia mobilalkalmazás.

A tesztelés célja: ellenőrizni, hogy minden definiált funkcionális követelmény teljesül, a rendszer stabilizáltan és megbízhatóan működik, a biztonsági mechanizmusok (HTTPS, JWT) helyesen működnek, és a szélsőséges bemeneti értékek sem okoznak váratlan hibákat.

Jelmagyarázat a tesztelési táblázatokban: ☒ PASS – a tesztet sikeresen teljesített (zöld háttér); **✗** FAIL – a tesztet sikertelen volt (piros háttér);  PARTIAL – részleges siker, kisebb eltérés (sárga háttér).

Szín	Jelentés
Zöld háttér (☒ PASS)	A tesztet az elvártan megfelelően lefutott
Piros háttér (✗ FAIL)	A tesztet hibát, nem várt viselkedést produkált
Sárga háttér ( PARTIAL)	Részleges siker – kisebb eltérés az elvárttól

2. Tesztelési stratégia

2.1 Alkalmazott tesztelési típusok

Típus	Eszköz	Célrendszer	Lefedett területek
Manuális funkcionális tesztelés	Chrome DevTools, böngésző	Web	UI, felhasználói folyamatok, validáció
API tesztelés (automata)	Jest + Supertest	Backend Node.js	REST végpontok, HTTP kódok, üzenetek
Unit tesztelés (automata)	MSTest (.NET)	WPF + Avalonia	OrdersClass.TotalPrice, ServerConnection metódusok
Biztonsági tesztelés	Postman + manuális	Backend + WPF	HTTPS kényszer, JWT validáció, jogosultságok
Reszponzív tesztelés	Chrome DevTools Device Mode	Web	Mobil, desktop nézetek

2.2 Tesztkörnyezet

Komponens	Konfiguráció
OS	Windows 11
Böngésző	Google Chrome (legújabb stabil)
Node.js	v20.x (LTS)
.NET SDK	.NET 8
Adatbázis (teszt)	SQLite in-memory (Jest) / MySQL teszt DB
API tesztelő	Postman + Jest/Supertest
Tesztelési időszak	2025. március – 2025. április

3. Web – Manuális tesztesetek

3.1 Regisztráció – manuális tesztek

A regisztrációs oldalon (/register) az alábbi validációs szabályok érvényesülnek: az e-mail mező kötelezően kell @ jelet tartalmazzon, az ÁSZF jelölőnégyzetet be kell pipálni, az összes kötelező mezőt ki kell tölteni.

TC#	Teszt neve	Bemeneti feltétel	Elvárt eredmény	Tényleges eredmény	Státusz
TC-W01	Sikeres regisztráció	Érvényes adatok megadása + ÁSZF bepipálva	Átírányítás /login-ra, sikeres visszajelzés	Átírányítás /login-ra, 201 Created	☒ PASS
TC-W02	E-mail @ jel nélkül	Email mező: 'teszt.domain.hu' (@ nélkül)	HTML5 validáció figyelmezteti a felhasználót	Böngésző beépített validáció: 'Kérjük, írjon @ jelet'	☒ PASS
TC-W03	ÁSZF nem pipálva	Adatok kitöltve, ÁSZF jelölőnégyzet kihagyva	Alert: 'El kell fogadnia az ÁSZF-et'	Alert megjelenik, regisztráció nem folytatódik	☒ PASS
TC-W04	Már regisztrált e-mail	Meglévő e-mail megadása	Alert: 'Van már ilyen felhasználó'	Szerver 409 Conflict, alert megjelenik	☒ PASS
TC-W05	Üres kötelező mező	Bármelyik kötelező mező üresen hagyva	Submit gomb disabled marad	Gomb inaktív, kérés nem kerül el	☒ PASS
TC-W06	Helyes e-mail formátum elfogadása	Email: 'felhasznalo@domain.hu'	Regisztráció folytatódik	Regisztráció sikeresen indul	☒ PASS

3.2 Bejelentkezés – manuális tesztek

TC#	Teszt neve	Bemeneti feltétel	Elvárt eredmény	Tényleges eredmény	Státusz
TC-W07	Sikeres bejelentkezés	Helyes felhasználónév + jelszó	JWT token sessionStorage-ban, átírányítás /profil-ra	Token eltárolódik, profiloldal betölt	☒ PASS
TC-W08	Helytelen jelszó	Helyes username, hibás jelszó	Alert: 'Hibás felhasználónév vagy jelszó'	401 Unauthorized, alert megjelenik	☒ PASS
TC-W09	Nem létező felhasználó	Nem regisztrált username megadása	Alert: felhasználó nem található	404 Not Found, hibaüzenet megjelenik	☒ PASS
TC-W10	Bejelentkezett → /login oldal	Bejelentkezve navigálás /login-ra	Automatikus átírányítás /profil-ra	auth-check.js átírányít, login nem jelenik meg	☒ PASS
TC-W11	Kijelentkezés	Kijelentkezés gomb megnyomása	Token törlődik, főoldalra irányítás	sessionStorage.clear(), átírányítás főoldalra	☒ PASS

3.3 Jogosultságvédelem – manuális tesztek

Az auth-check.js fájl minden oldalbetöltéskor ellenőrzi a token meglétét. Privát oldalak (profil, kosár, admin, stb.) token nélkül nem érhetőek el, és a rendszer a 404-es oldalra irányítja a felhasználót.

TC#	Teszt neve	Bemeneti feltétel	Elvárt eredmény	Tényleges eredmény	Státusz
TC-W12	Profil oldal token nélkül	Nincs token, /profil URL meglátogatása	Átirányítás /404 oldalra	auth-check.js átirányít, profil nem jelenik meg	☒ PASS
TC-W13	Kosár oldal token nélkül	Nincs token, /cart URL meglátogatása	Átirányítás /404 oldalra	Átirányítás megtörténik	☒ PASS
TC-W14	Admin panel normál userrel	Normál felhasználói token, /adminpanel URL	Átirányítás /404 oldalra	Admin jogkör hiánya, 404 jelenik meg	☒ PASS
TC-W15	Success oldal közvetlen elérése	Token nélkül /success URL	Átirányítás /404 oldalra	auth-check.js blokkolja a hozzáférést	☒ PASS
TC-W16	RecentOrders token nélkül	Nincs token, /recentorders URL	Átirányítás /404 oldalra	auth-check.js átirányít	☒ PASS

3.4 Kosár – manuális tesztek

TC#	Teszt neve	Bemeneti feltétel	Elvárt eredmény	Tényleges eredmény	Státusz
TC-W17	Termék kosárba helyezése	Bejelentkezve, termékoldal, Kosárba gomb	Termék megjelenik a kosárban, összeg frissül	localStorage frissül, toast megjelenik	☒ PASS
TC-W18	Mennyiség növelése	Kosárban 1 db termék, + gomb	Mennyiség 2-re nő, összeg frissül	Mennyiség és bruttó/nettó/ÁFA frissül	☒ PASS
TC-W19	Mennyiség csökkentése 0-ra	Kosárban 1 db termék, – gomb	Termék törlődik a kosárból	Termék eltűnik, üres kosár üzenet	☒ PASS
TC-W20	Max készlet túllépési kísérlet	Termék max. db-on van, + gomb	Alert: 'Ebből a termékből nincs több raktáron'	Alert megjelenik, mennyiség nem változik	☒ PASS
TC-W21	Összesítő számítás ellenőrzése	Kosárban 2 termék különböző árral	Bruttó = ár1+ár2, Nettó = Bruttó/1.27, ÁFA = különbség	Helyes számítás, helyesen jelenik meg	☒ PASS
TC-W22	Kosár több felhasználónál	Két különböző user bejelentkezik	Különálló kosarak (cart_{uid} kulcson)	Kosarak nem keverednek össze	☒ PASS

3.5 Rendelési folyamat – manuális tesztek

TC#	Teszt neve	Bemeneti feltétel	Elvárt eredmény	Tényleges eredmény	Státusz
TC-W23	Sikeres rendelés leadása	Token, cím beállítva, kosárban termék, kártya adatok megadva	201 Created, átirányítás /success-re	Rendelés DB-ben, success oldal betölt	☒ PASS
TC-W24	Rendelés üres kosárral	Token, kosár üres, Fizetés gomb	Alert: 'Üres a kosarad!'	Alert megjelenik, rendelés nem indul	☒ PASS
TC-W25	Rendelés szállítási cím nélkül	Token, nincs szállítási cím a DB-ben	Alert + Fizetés gomb inaktív	Alert és gomb disabled állapot	☒ PASS
TC-W26	Hiányos kártyaadatok	Token, cím OK, kártyaszám hiányzik	Alert: 'Kérlek add meg a bankkártya adatokat'	Validáció megakadályozza a küldést	☒ PASS
TC-W27	Success oldal adatai	Sikeres rendelés után /success megtekintése	Rendelési adatok helyesen megjelennek	sessionStorage lastPurchase adatai láthatók	☒ PASS
TC-W28	Kosár törlése rendelés után	Sikeres rendelés	localStorage kosár törlődik	cart_{uid} kulcs törlődik localStorage-ból	☒ PASS

3.6 Profil és cím kezelés – manuális tesztek

TC#	Teszt neve	Bemeneti feltétel	Elvárt eredmény	Tényleges eredmény	Státusz
TC-W29	Felhasználónév módosítása	Szerkesztés gomb, új név, pipa mentés	Adatbázisban frissül, oldalon megjelenik	200 OK, oldal frissül az új névvel	☒ PASS
TC-W30	Profilkép feltöltése (érvényes JPG)	Kép módosítása gomb, JPG kiválasztása	Kép feltöltődik, profilkép frissül	POST /uploadPFP 200 OK, kép megjelenik	☒ PASS
TC-W31	Jelszó módosítás helyes adatokkal	Jelenlegi jelszó + új jelszó + megerősítés	200 OK, sikeres üzenet	Jelszó frissül az adatbázisban	☒ PASS
TC-W32	Jelszó módosítás hibás jelenlegi jelszóval	Helytelen jelenlegi jelszó megadása	Alert: helytelen jelszó	401 Unauthorized, hibaüzenet	☒ PASS
TC-W33	Szállítási cím hozzáadása	Cím oldal, adatok kitöltése, mentés	Cím megjelenik a kosár oldalon is	POST /AddressRegister 201 Created	☒ PASS
TC-W34	Cím módosítása	Meglévő cím módosítása	Adatbázisban frissül	PUT /EditAddress 200 OK	☒ PASS
TC-W35	Cím megadása hiányos adatokkal	Csak város megadva, többi üres	400 Bad Request: 'Hiányzó adatok'	Szerver visszautasítja, hibaüzenet	☒ PASS

3.7 Reszponzivitás – manuális tesztek

TC#	Teszt neve	Bemeneti feltétel	Elvárt eredmény	Tényleges eredmény	Státusz
TC-W36	Főoldal – mobil (375px)	Chrome DevTools 375×667	Hero szekció látható, navigáció összezsukódik	Bootstrap grid szerint helyes	☒ PASS
TC-W37	Termékek – tablet (768px)	Chrome DevTools 768×1024	Kártyák kétszlopos elrendezés	Elrendezés helyes	☒ PASS
TC-W38	Kosár – mobil (375px)	Chrome DevTools 375×667	Kosárelemek egymás alá kerülnek	Elrendezés helyes	☒ PASS
TC-W39	Admin panel – asztali (1920px)	Teljes HD képernyő	Táblázat teljes szélességben jelenik meg	Admin UI teljes nézetben megjelenik	☒ PASS
TC-W40	Lábléc – mobil	375px szélességnél	Lábléc oszlopok egymás alá kerülnek	ft-col elemek 100% szélességre váltanak	☒ PASS

4. Backend – Automata API tesztek (Jest + Supertest)

Az API automata tesztek a Jest tesztelési keretrendszer és a Supertest HTTP tesztelő könyvtár segítségével valósultak meg. A tesztek valós adatbázis-műveletek végrehajtásával ellenőrzik az összes REST végpont viselkedését, státuszkódját és válaszüzenetét. A tesztek futtatásához szükséges a dbhandler.js konfiguráció, amely SQLite-ra van átállítva a tesztelési környezetben.

4.1 Felhasználói API tesztek

4.1.1 POST /UserLogin

```
it("should return 200 if the user successfully logs in", async () => {
  const res = await request(app)
    .post("/UserLogin")
    .send({ Username: "testuser", Password: "password123" })
  expect(res.status).toBe(200)
  expect(res.body).toHaveProperty("token")
})
```

AT#	Teszt neve	Bemeneti adat	Elvárt eredmény	Tényleges eredmény	Státusz
AT-01	POST /UserLogin – sikeres bejelentkezés	{ Username: 'testuser', Password: 'password123' }	200 OK + { token: '...' }	200 OK + token property	☒ PASS
AT-02	GET /getUser – valid tokennel	Bearer validToken fejléccel	200 OK + { Username: 'testuser', Email: ... }	200 OK, helyes felhasználói adatok	☒ PASS

4.1.2 POST /UserRegister

```
it("should return 201 on successful registration", async () => {
  const res = await request(app)
    .post("/UserRegister")
    .send({ Username: "newuser", Email: "newuser@example.com", Password: "password123" })
  expect(res.status).toBe(201)
  expect(res.body.message).toBe("Sikeres regisztráció")
})
it("should return 409 if user already exists", async () => {
  // ... user already created ...
  expect(res.status).toBe(409)
  expect(res.body.message).toBe("Van már ilyen felhasználó")
})
```

AT#	Teszt neve	Bemeneti adat	Elvárt eredmény	Tényleges eredmény	Státusz
AT-03	POST /UserRegister – sikeres regisztráció	{ Username, Email, Password } érvényes adatokkal	201 Created + { message: 'Sikeres regisztráció' }	201 + helyes üzenet	☒ PASS
AT-04	POST /UserRegister – már létező user	Meglévő Username + Email	409 Conflict + { message: 'Van már ilyen felhasználó' }	409 + helyes üzenet	☒ PASS

4.1.3 PUT /EditUser

AT#	Teszt neve	Bemeneti adat	Elvárt eredmény	Tényleges eredmény	Státusz
AT-05	PUT /EditUser – sikeres módosítás	{ Username, Email, Password, OldPassword } mind érvényes	200 OK + { message: 'Sikeres adatmódosítás!' }	200 OK + helyes üzenet	☒ PASS
AT-06	PUT /EditUser – nincs módosítandó adat	Üres body {}	400 Bad Request + { message: 'Nincs módosítandó adat.' }	400 + helyes üzenet	☒ PASS

4.2 Cím API tesztek (Address)

```
it("should create a new address", async () => {
  const res = await request(app)
    .post("/AddressRegister")
    .set("Authorization", `Bearer ${validToken}`)
    .send({ City: "Budapest", Zip: 1111, Address1: "Teszt utca 1" })
  expect(res.status).toBe(201)
  expect(res.body.message).toBe("Sikeres Lakhely regisztráció")
})
```

AT#	Teszt neve	Bemeneti adat	Elvárt eredmény	Tényleges eredmény	Státusz
AT-07	POST /AddressRegister – sikeres létrehozás	{ City, Zip, Address1 } + auth token	201 Created + { message: 'Sikeres Lakhely regisztráció' }	201 + helyes üzenet	☒ PASS
AT-08	GET /AddressGets – cím lekérdezése	Érvényes auth token, létező cím	200 OK + { City: 'Budapest' }	200 + helyes City mező	☒ PASS
AT-09	GET /AddressGets – nincs cím	Érvényes auth token, nincs cím DB-ben	404 Not Found + { message: 'Nincs ilyen Lakhely' }	404 + helyes üzenet	☒ PASS
AT-10	PUT /EditAddress – sikeres módosítás	{ City, Zip, Address1 } + auth token	200 OK + { message: 'Sikeres módosítás' }	200 + helyes üzenet	☒ PASS
AT-11	PUT /EditAddress – hiányzó adatok	Csak { City } mező	400 Bad Request + { message: 'Hiányzó adatok' }	400 + helyes üzenet	☒ PASS
AT-12	PUT /EditAddress – nem létező cím	Nincs cím DB-ben	404 Not Found + { message: 'Nincs mentett cím ehhez a felhasználóhoz' }	404 + helyes üzenet	☒ PASS

4.3 Számlázási cím API tesztek (BillingAddress)

AT#	Teszt neve	Bemeneti adat	Elvárt eredmény	Tényleges eredmény	Státusz
AT-13	POST /BillingAddressRegister – sikeres	{ City, Zip, Address1 } + auth	201 Created + { message: 'Sikeres számlázási cím regisztráció' }	201 + helyes üzenet	☒ PASS
AT-14	GET /BillingAddressGets – lekérdezés	Érvényes token, létező számlázási cím	200 OK + { City: 'Debrecen' }	200 + helyes adatok	☒ PASS
AT-15	GET /BillingAddressGets – nincs cím	Nincs számlázási cím	404 + { message: 'Nincs ilyen számlázási cím' }	404 + helyes üzenet	☒ PASS
AT-16	PUT /EditBillingAddress – sikeres	{ City, Zip, Address1 } + auth	200 OK + { message: 'Sikeres módosítás' }	200 + helyes üzenet	☒ PASS
AT-17	PUT /EditBillingAddress – hiányzó adatok	{ City } csak	400 + { message: 'Hiányzó adatok' }	400 + helyes üzenet	☒ PASS
AT-18	PUT /EditBillingAddress – nincs cím	Nincs cím DB-ben	404 + { message: 'Nincs mentett számlázási cím' }	404 + helyes üzenet	☒ PASS

4.4 Rendelés API tesztek

```
it("should create a new order", async () => {
  const res = await request(app)
    .post("/createOrder")
    .set("Authorization", `Bearer ${validToken}`)
    .send({
      AddressId: addressId,
      cart: [{ Id: productId, Name: "Teszt termék", Price: 1000, quantity: 2
    }]
  })
  expect(res.status).toBe(201)
  expect(res.body.message).toBe("Rendelés sikeres!")
  const product = await dbhandler.Products.findByPk(productId)
  expect(product.Ammount).toBe(8)
})
```

AT#	Teszt neve	Bemeneti adat	Elvárt eredmény	Tényleges eredmény	Státusz
AT-19	POST /createOrder – sikeres rendelés	{ AddressId, cart: [{ Id, Price, quantity: 2 }] }	201 + { message: 'Rendelés sikeres!', orderId: N } + készlet csökkentve (10→8)	201 + orderId + Ammount=8	☒ PASS
AT-20	POST /createOrder – üres kosár	{ AddressId, cart: [] }	400 + { message: 'Üres a kosár' }	400 + helyes üzenet	☒ PASS
AT-21	POST /createOrder – hiányzó AddressId	{ cart: [...] } – AddressId nélkül	400 + { message: 'Hiányzó cím azonosító' }	400 + helyes üzenet	☒ PASS
AT-22	GET /getMyOrders – rendelések lekérdezése	Érvényes token, van rendelés	200 + tömb, OrderItems és Address mezőkkel	200 + helyes struktúra	☒ PASS

5. WPF – Unit tesztek (MSTest)

A WPF projekt MSTest alapú unit teszteket tartalmaz (Notrox_WPF_Test/Test1.cs). A tesztek két fő területet fednek le: az OrdersClass.TotalPrice számítási logikáját, és a ServerConnection osztály HTTP kommunikációját.

5.1 OrdersClass.TotalPrice unit tesztek

Az OrdersClass egy computed property-t tartalmaz: TotalPrice = Items?.Sum(i => i.PriceAtPurchase * i.Quantity) ?? 0. Az alábbi tesztek ezt a számítási logikát validálják különböző szélsőértékeken.

```
[TestMethod]
public void OrderTotalPriceNull()
{
    OrdersClass order = new OrdersClass();
    order.Items = new();
    Assert.AreEqual(0, order.TotalPrice);
}

[TestMethod]
public void OrderTotalPriceOneItem()
{
    OrdersClass order = new OrdersClass();
    order.Items = new List<OrderItemClass> {
        new OrderItemClass { ProductId = 1, Quantity = 1, PriceAtPurchase =
200 }
    };
    Assert.AreEqual(200, order.TotalPrice);
}
```

UT#	Metódus neve	Bemeneti feltétel	Elvárt kimenet	Tényleges kimenet	Státusz
UT-WPF-01	OrderTotalPriceNull	Üres Items lista (new())	TotalPrice == 0	Assert.AreEqual(0, order.TotalPrice) – PASS	☒ PASS
UT-WPF-02	OrderTotalPriceOneItem	1 db termék, Quantity=1, Price=200	TotalPrice == 200	Assert.AreEqual(200, order.TotalPrice) – PASS	☒ PASS
UT-WPF-03	OrderTotalPriceManyItems	5 db termék, mindegyik Quantity=1, Price=200	TotalPrice == 1000	Assert.AreEqual(1000, order.TotalPrice) – PASS	☒ PASS
UT-WPF-04	OrderTotalPrice_WithQuantities	Product1: Qty=2, Price=100; Product2: Qty=3, Price=50	TotalPrice == 350	Assert.AreEqual(350, order.TotalPrice) – PASS	☒ PASS
UT-WPF-05	OrderTotalPrice_ZeroQuantity	1 db termék, Quantity=0, Price=999	TotalPrice == 0	Assert.AreEqual(0, order.TotalPrice) – PASS	☒ PASS
UT-WPF-06	OrderTotalPrice_NegativePrice	1 db termék, Quantity=1, Price=-100	TotalPrice == -100	Assert.AreEqual(-100, order.TotalPrice) – PASS	☒ PASS
UT-WPF-07	OrderTotalPrice_LargeValues	1 db termék, Quantity=int.MaxValue, Price=1	TotalPrice == int.MaxValue	Assert.AreEqual(int.MaxValue, ...) – PASS	☒ PASS

5.2 ServerConnection – biztonsági és HTTP unit tesztek

A `ServerConnection` osztály konstruktora `ArgumentException`-t dob HTTP URL esetén. A hálózati hívások metódusai hibakezelés nélkül nem dobnak kivételt, hanem `null/false/üres` listával térnek vissza.

```
[TestMethod]
public async Task TestConnectionUrl()
{
    Assert.ThrowsException<ArgumentException>(() => new
    ServerConnection("asd"));
}

[TestMethod]
public void Constructor_Http_ShouldThrow()
{
    Assert.ThrowsException<ArgumentException>(
        () => new ServerConnection("http://example.com"));
}
```

UT#	Metódus neve	Bemeneti feltétel	Elvárt kimenet	Tényleges kimenet	Státusz
UT-WPF-08	TestConnectionUrl	HTTP-nélküli URL: 'asd'	ArgumentException dobódik	ThrowsException <ArgumentException> – PASS	☒ PASS
UT-WPF-09	Constructor_Http_ShouldThrow	URL: 'http://example.com' (HTTP, nem HTTPS)	ArgumentException dobódik	ThrowsException <ArgumentException> – PASS	☒ PASS
UT-WPF-10	LoginGoodHttps	HTTPS URL, hibás szerver cím	Login visszatér false-szal	Assert.IsFalse(worked) – PASS	☒ PASS
UT-WPF-11	Login_EmptyCredentials	Üres username és jelszó	false visszatérés (nincs kivétel)	Assert.IsFalse(result) – PASS	☒ PASS
UT-WPF-12	Login_NullCredentials	Null username és jelszó	false visszatérés	Assert.IsFalse(result) – PASS	☒ PASS
UT-WPF-13	GetCurrentUser_Failure_ReturnsNull	Nem elérhető szerver	null visszatérés	Assert.IsNull(user) – PASS	☒ PASS
UT-WPF-14	ListUsers_Failure_ReturnsEmpty	Nem elérhető szerver	Nem null, üres lista	Assert.AreEqual(0, users.Count) – PASS	☒ PASS
UT-WPF-15	DeleteUser_Failure_ReturnsFalse	Nem elérhető szerver	false visszatérés	Assert.IsFalse(result) – PASS	☒ PASS
UT-WPF-16	AddProduct_Failure_ReturnsFalse	Nem elérhető szerver	false visszatérés	Assert.IsFalse(result) – PASS	☒ PASS
UT-WPF-17	DeleteProduct_Failure_ReturnsFalse	Nem elérhető szerver	false visszatérés	Assert.IsFalse(result) – PASS	☒ PASS
UT-WPF-18	EditProduct_Failure_ReturnsFalse	Nem elérhető szerver	false visszatérés	Assert.IsFalse(result) – PASS	☒ PASS
UT-WPF-19	EditOrder_Failure_ReturnsFalse	Nem elérhető szerver	false visszatérés	Assert.IsFalse(result) – PASS	☒ PASS
UT-WPF-20	EditAddress_Failure_ReturnsFalse	Nem elérhető szerver	false visszatérés	Assert.IsFalse(result) – PASS	☒ PASS
UT-WPF-21	EditBillingAddress_Failure_ReturnsFalse	Nem elérhető szerver	false visszatérés	Assert.IsFalse(result) – PASS	☒ PASS
UT-WPF-22	MultipleCalls_ShouldNotThrow	10x ListUsers + 10x ListProducts	Kivétel nem keletkezik	Assert.IsTrue(true) – PASS	☒ PASS
UT-WPF-23	Logout_MultipleCalls	3x Logout() hívás egymás után	Kivétel nem keletkezik	Assert.IsTrue(true) – PASS	☒ PASS
UT-WPF-24	Methods_ShouldNeverThrow	Login + ListUsers + ListProducts + GetCurrentUser egymás után	Kivétel nem keletkezik	Assert.IsTrue(true) – PASS	☒ PASS

6. Avalonia – Unit tesztek (MSTest)

Az Avalonia projekt unit tesztjei (Notrox_Test/Test1.cs) megegyeznek a WPF tesztekkel, egy különbséggel: a Throws szintaxisa Assert.Throws<T>() Avaloniában, szemben a WPF Assert.ThrowsException<T>() szintaxissal.

6.1 OrdersClass.TotalPrice tesztek – Avalonia

Az Avalonia projekt OrdersClass implementációja ugyanolyan TotalPrice számítást tartalmaz, mint a WPF verzió. A tesztek azonos eredményeket produkálnak.

UT #	Metódus neve	Bemeneti feltétel	Elvárt kimenet	Tényleges kimenet	Státusz
UT-AV-01	OrderTotalPriceNull (Avalonia)	Üres Items lista	TotalPrice == 0	Assert.AreEqual(0, ...) – PASS	☒ PASS
UT-AV-02	OrderTotalPriceOneItem (Avalonia)	1 db, Qty=1, Price=200	200	Assert.AreEqual(200, ...) – PASS	☒ PASS
UT-AV-03	OrderTotalPriceManyItems (Avalonia)	5 db, mindegyik Qty=1, Price=200	1000	Assert.AreEqual(1000, ...) – PASS	☒ PASS
UT-AV-04	OrderTotalPrice_WithQuantities (Avalonia)	Qty=2×100 + Qty=3×50	350	Assert.AreEqual(350, ...) – PASS	☒ PASS
UT-AV-05	OrderTotalPrice_ZeroQuantity (Avalonia)	Qty=0, Price=999	0	Assert.AreEqual(0, ...) – PASS	☒ PASS
UT-AV-06	OrderTotalPrice_NegativePrice (Avalonia)	Qty=1, Price=-100	-100	Assert.AreEqual(-100, ...) – PASS	☒ PASS
UT-AV-07	OrderTotalPrice_LargeValues (Avalonia)	Qty=int.MaxValue, Price=1	int.MaxValue	Assert.AreEqual(int.MaxValue, ...) – PASS	☒ PASS

6.2 ServerConnection tesztek – Avalonia

Megjegyzés: Az Avalonia tesztek `Assert.Throws<T>()` szintaxist alkalmaznak (MSTest Avalonia kompatibilis verzió), szemben a WPF `Assert.ThrowsException<T>()` szintaxissal. A tesztlogika és az eredmények azonosak.

UT#	Metódus neve	Bemeneti feltétel	Elvárt kimenet	Tényleges kimenet	Státusz
UT-AV-08	TestConnectionUrl (Avalonia)	'asd' URL	ArgumentException	Assert.Throws<ArgumentException> – PASS	☒ PASS
UT-AV-09	Constructor_Http (Avalonia)	'http://example.com'	ArgumentException	Assert.Throws<ArgumentException> – PASS	☒ PASS
UT-AV-10	LoginGoodHttps (Avalonia)	HTTPS + hibás szerver	false visszatérés	Assert.IsFalse(worked) – PASS	☒ PASS
UT-AV-11..24	Összes ServerConnection teszt (Avalonia)	Azonos WPF tesztekkel	Azonos eredmények WPF-fel	Minden teszt PASS	☒ PASS

7. Tesztelési összefoglaló és statisztikák

7.1 Összes tesztet összesítője

Tesztkategória	Tesztek száma	☑ PASS	✗ FAIL	Sikerességi arány
Web – Manuális (TC-W01..W40)	40 db	40	0	100%
Backend API – Jest/Supertest (AT-01..22)	22 db	22	0	100%
WPF – MSTest Unit (UT-WPF-01..24)	24 db	24	0	100%
Avalonia – MSTest Unit (UT-AV-01..24)	24 db	24	0	100%
Összesen	110 db	110	0	100%

7.2 Azonosított és javított hibák

Hiba ID	Leírás	Súlyosság	Javítás státusza
BUG-01	Kosárkulcs ütközés: különböző userek kosara felülírta egymást → cart_{uid} megoldás	Kritikus	☑ Javítva
BUG-02	auth-check.js nem ellenőrizte az admin jogkört, normál user beléphetett adminpanelre	Kritikus	☑ Javítva
BUG-03	Termék törlése után a lista nem frissült automatikusan	Közepes	☑ Javítva
BUG-04	Kártyalejárat: 00 hónapot engedett meg	Kisebb	☑ Javítva
BUG-05	ServerConnection HTTP URL-nél nem dobott kivételt (hiányzó validáció)	Kritikus	☑ Javítva
BUG-06	OrdersClass.TotalPrice null Items esetén NullReferenceException-t dobott	Kritikus	☑ Javítva

7.3 Következtetések

A tesztelési folyamat eredményei alapján a NOTROX platform mindhárom kliensalkalmazása a definiált követelményeket teljesíti. A 110 tesztet 100%-os sikerességi arányt mutat. Az azonosított 6 kritikus és kisebb hiba az éles kiadás előtt javításra került.

A WPF és Avalonia MSTest tesztek különösen fontos visszajelzést adtak a ServerConnection osztály robusztusságáról: a metódusok minden hálózati hiba esetén kivétel

helyett null/false/üres listával térnek vissza, ami megakadályozza az alkalmazás váratlan összeomlását.

8. Összefoglalás

A NOTROX platform tesztelési dokumentációja átfogóan lefedi a szoftver három klienskomponensének összes tesztelési tevékenységét. A manuális web tesztek a felhasználói folyamatok részletes validációját biztosítják, az automata Jest/Supertest tesztek a backend API megbízhatóságát igazolják, az MSTest unit tesztek pedig a kritikus üzleti logika (rendelés összesítő számítás) és az infrastrukturális kód (HTTP kapcsolat, biztonság) helyes működését garantálják.

A jövőbeli fejlesztési tervekben szerepel az automata teszt lefedettség növelése Playwright End-to-End tesztekkel, amelyek az egész felhasználói folyamatot (regisztráció → bejelentkezés → vásárlás) szimulálják valós böngészős környezetben.

Felhasznált irodalom és forrásmegjelölés

1. Jest dokumentáció. Elérhető: <https://jestjs.io/docs/getting-started> (Letöltve: 2025. március)
2. Supertest npm dokumentáció. Elérhető: <https://github.com/ladjs/supertest> (Letöltve: 2025. március)
3. MSTest dokumentáció. Elérhető: <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-with-mstest> (Letöltve: 2025)
4. Playwright E2E tesztelési keretrendszer. Elérhető: <https://playwright.dev/> (Letöltve: 2025)
5. ISTQB tesztelési alapok. Elérhető: <https://www.istqb.org/> (Letöltve: 2025)